

Math Foundations Textbook Pilot

Data Analysis Textbook Project

Chapters 1–12 Pilot Build

Contents

1	Logic and Deductive Reasoning	1
1.1	Statements and Logical Connectives	1
1.2	Inclusive Or and Truth Tables	2
1.3	Valid Arguments and Counterexamples	2
1.4	Translating Logical Formulas	3
1.5	Tautologies, Contradictions, and Logical Equivalence	3
1.6	De Morgan's Laws	4
1.7	Implication	4
1.8	Biconditional Statements	5
1.9	Chapter Summary	6
2	Vectors in $\mathbb{R}^n$	7
2.1	The Space $\mathbb{R}^n$	7
2.2	Entries, Coordinates, and Subvectors	8
2.3	Indexing Conventions	8
2.4	Special Vectors	8
2.5	Sparse and Dense Vectors	9
2.6	Vectors as Data and Measurements	9
2.7	Probability Vectors and Time Series	10
2.8	Chapter Summary	10
3	Vector Operations and Linear Combinations	12
3.1	Vector Addition	12
3.2	Vector Subtraction and Differences	13
3.3	A Notational Warning: Vectors and Scalars	13
3.4	Scalar Multiplication	14
3.5	Linear Combinations	14
3.6	Standard Basis Representation	14
3.7	Lines and Line Segments	15

3.8	Data Analysis Bridge	15
3.9	Chapter Summary	16
4	Sets, Set Operations, and Quantifiers	17
4.1	Sets and Membership	17
4.2	Set Operations	18
4.3	De Morgan's Laws for Sets	18
4.4	Subsets, Equality, and Special Sets	19
4.5	Families of Sets and a Finite Example	19
4.6	Common Number Systems	20
4.7	Quantified Statements	20
4.8	Negating Quantified Statements	20
4.9	Data Analysis Bridge	21
4.10	Chapter Summary	21
5	Dot Products, Norms, Distance, Angles, and Orthogonality	22
5.1	The Inner Product	22
5.2	Properties of the Inner Product	23
5.3	Inner Products as Selection, Sums, Means, and Weighted Sums	23
5.4	Computational Cost of Vector Operations	24
5.5	The Euclidean Norm	24
5.6	Norm Properties	24
5.7	Distance Between Vectors	25
5.8	Angles and Orthogonality	25
5.9	Example: Earth Geometry	25
5.10	Data Analysis Bridge	26
5.11	Chapter Summary	26
6	Linear Independence, Span, Bases, and Dimension	28
6.1	Span: All Vectors Built from a Collection	28
6.2	Linear Dependence	29
6.3	Linear Independence	29
6.4	How Many Independent Vectors Can There Be?	30
6.5	Bases	30
6.6	Coordinates in a Basis Are Unique	31
6.7	The Standard Basis	31
6.8	Orthonormal Bases	32
6.9	Data Analysis Bridge	32

6.10 Chapter Summary	32
7 Matrices and Matrix-Vector Multiplication	34
7.1 Matrices as Rectangular Arrays	34
7.2 Rows, Columns, Blocks, and Submatrices	35
7.3 Special Matrices	35
7.4 Transpose and Symmetry	36
7.5 Matrix Arithmetic	37
7.6 Matrix-Vector Multiplication	37
7.6.1 Row Inner-Product View	38
7.6.2 Column Linear-Combination View	38
7.7 Computational Cost and Shape Checking	38
7.8 A Matrix-Vector Dynamics Example	39
7.9 Data Analysis Bridge	40
7.10 Chapter Summary	40
8 Linear Systems and Vandermonde Matrices	42
8.1 Linear Systems in Matrix Form	42
8.2 Triangular Systems	43
8.3 Back Substitution	44
8.4 Forward Substitution	44
8.5 Solving Systems by Reducing to Triangular Form	45
8.6 Polynomial Interpolation as a Linear System	45
8.7 Vandermonde Matrices	46
8.8 Exact Systems and Approximate Systems	47
8.9 Data Analysis Bridge	47
8.10 Chapter Summary	48
9 Orthogonal Bases, Gram-Schmidt, and QR	49
9.1 Orthogonal and Orthonormal Sets	49
9.2 The Goal of Gram-Schmidt	50
9.3 Removing Previously Used Directions	51
9.4 The Gram-Schmidt Algorithm	51
9.5 Why the New Vector Is Orthogonal to the Old Ones	52
9.6 A Two-Vector Example	52
9.7 Matrices with Orthonormal Columns	53
9.8 Tall or Square Matrices with Independent Columns	54
9.9 From Gram-Schmidt to QR Factorization	54

9.10 Using QR to Solve a Linear System	55
9.11 Orthogonal-Polynomial Intuition	55
9.12 Data Analysis Bridge	56
9.13 Chapter Summary	56
10 Matrix Inverses, Rank, and Pseudoinverses	57
10.1 Left Inverses	57
10.2 Right Inverses	58
10.3 Square Matrix Inverses	59
10.4 Computing a Square Inverse with QR	59
10.5 Rank and Full Rank	60
10.6 Why $A^T A$ Appears	60
10.7 Full-Rank Pseudoinverse Formulas	61
10.8 Computing Full-Rank Pseudoinverses with QR	61
10.9 Rank Determination with QR	62
10.10 Data Analysis Bridge	62
10.11 Chapter Summary	62
11 Subspaces: Column Space, Row Space, and Null Space	64
11.1 Subspaces	64
11.2 Column Space	65
11.3 Row Space	65
11.4 Null Space	66
11.5 Left Null Space	66
11.6 Orthogonality Among the Subspaces	67
11.7 The Four Fundamental Subspaces	67
11.8 Example: Non-Uniqueness in a Targeting Problem	68
11.9 Data Analysis Bridge	68
11.10 Chapter Summary	69
12 Multivariable Functions, Gradients, and Least Squares	70
12.1 Multivariable Functions	70
12.2 Partial Derivatives	71
12.3 Directional Derivatives	72
12.4 Differentiability, Derivatives, and the Gradient	72
12.5 Jacobians for Vector-Valued Functions	73
12.6 Derivative Rules	73
12.7 The Chain Rule	74

12.8 Stationary Points	75
12.9 The Least-Squares Problem	76
12.10Expanding the Least-Squares Objective	76
12.11Gradient and the Normal Equations	76
12.12Residual Orthogonality and Projection	77
12.13A Chain-Rule Application: Neural-Network Loss	78
12.14Data Analysis Bridge	79
12.15Chapter Summary	79

Chapter 1

Logic and Deductive Reasoning

Mathematical foundations for operations research begin with the language of careful reasoning. Before we solve systems of equations, analyze vectors, optimize functions, or interpret least-squares estimates, we need to be precise about what a statement means and what conclusions follow from a set of assumptions.

This course is built around three pillars:

1. set theory and logic,
2. linear algebra, and
3. multivariate differential calculus.

Logic comes first because it gives us the grammar for mathematical arguments. In later chapters, assumptions will play the role of premises. Conclusions are only as reliable as the assumptions and reasoning that support them.

1.1 Statements and Logical Connectives

A *statement* is a sentence that has a truth value: it is either true or false. For example, “8 is prime” is a statement, and so is “8 is composite.” In mathematical logic, we often replace statements with symbols so that we can focus on the structure of an argument rather than the specific subject matter.

Let

$$P : \text{“8 is prime”}, \quad Q : \text{“8 is composite”}.$$

The English statement “8 is either prime or composite” can then be written symbolically as

$$P \vee Q.$$

Here $\vee$ means *or*. We also use $\wedge$ for *and* and $\neg$ for *not*. These are called *logical connectives* because they combine simpler statements into compound statements.

Symbol	Meaning
$P \vee Q$	P or Q
$P \wedge Q$	P and Q
$\neg P$	not P

The negation $\neg P$ reverses the truth value of P . If P is true, then $\neg P$ is false. If P is false, then $\neg P$ is true.

1.2 Inclusive Or and Truth Tables

In mathematics, the word “or” is usually *inclusive*. This means that $P \vee Q$ is true when P is true, when Q is true, or when both are true. This differs from exclusive-or, which means “one or the other, but not both.” Exclusive-or is sometimes written xor, but it is much less common in the logic used in this course.

One way to verify a logical argument is to build a *truth table*. A truth table lists all possible truth values of the basic statements and then computes the truth values of the compound statements.

For example, consider the argument

$$P \vee Q, \quad \neg P, \quad \therefore Q.$$

In words: if either P or Q is true, and P is not true, then Q must be true. This is the same structure as the argument

Every natural number greater than 1 is either prime or composite. The number 8 is not prime. Therefore, 8 is composite.

It is also the same structure as

Every natural number is either even or odd. The number 7 is not divisible by 2. Therefore, 7 is odd.

The truth table for the relevant statements is

P	Q	$\neg P$	$P \vee Q$
F	F	T	F
F	T	T	T
T	F	F	T
T	T	F	T

The only row in which both premises $P \vee Q$ and $\neg P$ are true is the second row. In that row, Q is also true. Therefore, the argument is valid.

1.3 Valid Arguments and Counterexamples

A logical argument is *valid* if, whenever all of its premises are true, its conclusion must also be true. Validity is about the structure of the reasoning, not about whether the particular real-world statements are interesting or important.

To show that an argument is *not* valid, it is enough to find one counterexample: one assignment of truth values that makes every premise true while making the conclusion false.

Consider the argument

Either the butler or the maid is guilty. Either the maid or the cook is guilty. Therefore, either the butler or the cook is guilty.

Let

P : “the butler is guilty”, Q : “the maid is guilty”, R : “the cook is guilty”.

The argument becomes

$$P \vee Q, \quad Q \vee R, \quad \therefore P \vee R.$$

The truth table is

P	Q	R	$P \vee Q$	$Q \vee R$	$P \vee R$
F	F	F	F	F	F
F	F	T	F	T	T
F	T	F	T	T	F
F	T	T	T	T	T
T	F	F	T	F	T
T	F	T	T	T	T
T	T	F	T	T	T
T	T	T	T	T	T

The third row is a counterexample. In that row, P is false, Q is true, and R is false. Both premises $P \vee Q$ and $Q \vee R$ are true, but the conclusion $P \vee R$ is false. Therefore, the argument is not valid.

1.4 Translating Logical Formulas

Symbolic notation is useful only if we can translate between symbols and ordinary language. Let

S : “John is stupid”, L : “John is lazy”.

Then the following formulas have different logical structures:

Formula	Meaning
$(\neg S \wedge L) \vee S$	either John is not stupid and lazy, or John is stupid
$\neg S \vee (L \vee S)$	John is not stupid, or John is lazy, or John is stupid
$\neg(S \wedge L) \vee S$	either John is not both stupid and lazy, or John is stupid

The exact English sentence is less important than the structure. Parentheses show which statements are grouped together. The outermost connective determines the main logical relationship.

1.5 Tautologies, Contradictions, and Logical Equivalence

A logical formula is a *tautology* if it is true for every assignment of truth values. A simple example is

$$P \vee \neg P.$$

Either P is true, or it is not true. There is no third possibility in classical logic.

A logical formula is a *contradiction* if it is false for every assignment of truth values. A simple example is

$$P \wedge \neg P.$$

A statement and its negation cannot both be true at the same time.

Two logical formulas are *logically equivalent* if they have the same truth value for every possible assignment of truth values. We write logical equivalence with $\equiv$. For example, if two formulas are equivalent, they can be substituted for one another inside a larger logical argument without changing the argument's truth structure.

1.6 De Morgan's Laws

De Morgan's laws are two important logical equivalences involving negation, *and*, and *or*:

$$\neg(P \wedge Q) \equiv \neg P \vee \neg Q,$$

$$\neg(P \vee Q) \equiv \neg P \wedge \neg Q.$$

The first law says that “not both” is equivalent to “at least one is not.” The second law says that “neither” is equivalent to “not the first and not the second.”

These laws are easy to state but important to use correctly. For example, the negation of

$$P \wedge Q$$

is not

$$\neg P \wedge \neg Q.$$

The correct negation is

$$\neg P \vee \neg Q.$$

This distinction matters whenever we negate assumptions or construct complements of sets.

A truth table can verify the first law:

P	Q	$P \wedge Q$	$\neg(P \wedge Q)$	$\neg P$	$\neg P \vee \neg Q$
T	T	T	F	F	F
T	F	F	T	F	T
F	T	F	T	T	T
F	F	F	T	T	T

The columns for $\neg(P \wedge Q)$ and $\neg P \vee \neg Q$ match in every row, so the formulas are logically equivalent.

1.7 Implication

A conditional statement, or *implication*, has the form

$$P \Rightarrow Q.$$

This is read as “if P , then Q ,” or “ P implies Q .” The statement P is the premise or antecedent, and Q is the conclusion or consequent.

For example,

If I leave in 10 minutes, then I will be late.

can be represented by

$$P \Rightarrow Q,$$

where P is “I leave in 10 minutes” and Q is “I will be late.”

The truth table for implication is

P	Q	$P \Rightarrow Q$
F	F	T
F	T	T
T	F	F
T	T	T

An implication is false only in the case where the premise P is true and the conclusion Q is false. If the premise is false, then the implication is true as a statement of logic. This convention may feel strange at first, but it gives implication the behavior needed for mathematical arguments.

One useful way to remember implication is:

To disprove “if P , then Q ,” find a case where P is true but Q is false.

That case is the counterexample.

1.8 Biconditional Statements

A biconditional statement has the form

$$P \Leftrightarrow Q.$$

It is read as “ P if and only if Q ,” often abbreviated as “ P iff Q .” The biconditional is true exactly when P and Q have the same truth value.

Equivalently,

$$P \Leftrightarrow Q$$

has the same truth values as

$$(P \Rightarrow Q) \wedge (Q \Rightarrow P).$$

Thus, to prove a biconditional, one usually proves two implications:

1. if P , then Q ;
2. if Q , then P .

The truth table is

P	Q	$P \Leftrightarrow Q$
F	F	T
F	T	F
T	F	F
T	T	T

The biconditional is stronger than implication. The statement $P \Rightarrow Q$ says that P is sufficient for Q . The statement $P \Leftrightarrow Q$ says that each statement implies the other.

1.9 Chapter Summary

This chapter introduced the logical language used throughout the rest of the text. The main ideas are:

- A statement is a sentence that is either true or false.
- The basic logical connectives are $\vee$, $\wedge$, and $\neg$.
- Mathematical $\vee$ is inclusive unless stated otherwise.
- A truth table checks all possible assignments of truth values.
- An argument is valid if true premises always force a true conclusion.
- A counterexample is one assignment where the premises are true and the conclusion is false.
- A tautology is always true; a contradiction is always false.
- De Morgan's laws explain how negation interacts with $\vee$ and $\wedge$.
- An implication $P \Rightarrow Q$ fails only when P is true and Q is false.
- A biconditional $P \Leftrightarrow Q$ means that P and Q imply each other.

The next chapter begins the linear algebra portion of the course by introducing vectors in $\mathbb{R}^n$.

Chapter 2

Vectors in $\mathbb{R}^n$

The next part of the course begins the linear algebra pillar. The basic object is a vector. Vectors will later be added, scaled, multiplied through dot products, collected into matrices, and used to describe regression geometry. Before doing any of that, we need a clear answer to a simpler question: what is a vector?

In this chapter, a vector is an ordered collection of real numbers. The entries might represent coordinates in physical space, but they might also represent inventory amounts, probabilities, measurements over time, or values of a variable in a data set. The important point is that the entries are organized and interpreted together.

2.1 The Space $\mathbb{R}^n$

Let $\mathbb{R}$ denote the set of real numbers. For a positive integer n , the set $\mathbb{R}^n$ is the set of all ordered lists of n real numbers. An element of $\mathbb{R}^n$ is called an n -vector. Unless stated otherwise, vectors are written as columns:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^n.$$

The same vector is sometimes written on one line as

$$\mathbf{x} = (x_1, x_2, \dots, x_n)^T,$$

where the superscript T indicates transpose. The transpose notation reminds us that the displayed list $(x_1, x_2, \dots, x_n)$ is being treated as a column vector.

The number n is the *dimension* or *length* of the vector. A scalar is a single number; a vector is a structured collection of numbers. For example,

$$\begin{bmatrix} 5 \\ 2 \\ 9 \end{bmatrix} \in \mathbb{R}^3$$

is a 3-vector, while $5 \in \mathbb{R}$ is a scalar.

2.2 Entries, Coordinates, and Subvectors

For a vector $\mathbf{x} \in \mathbb{R}^n$, the scalar values

$$x_1, x_2, \dots, x_n$$

are called the *entries*, *components*, *elements*, or *coordinates* of $\mathbf{x}$. The notation x_i means the i th entry of $\mathbf{x}$.

It is often useful to build a larger vector by stacking smaller vectors. Suppose

$$\mathbf{b} \in \mathbb{R}^m, \quad \mathbf{c} \in \mathbb{R}^n, \quad \mathbf{d} \in \mathbb{R}^p.$$

Then the stacked vector

$$\mathbf{a} = \begin{bmatrix} \mathbf{b} \\ \mathbf{c} \\ \mathbf{d} \end{bmatrix}$$

belongs to $\mathbb{R}^{m+n+p}$. In this situation, $\mathbf{b}$, $\mathbf{c}$, and $\mathbf{d}$ are *subvectors* or *slices* of $\mathbf{a}$.

Colon notation is often used to describe consecutive entries. If $r \leq s$, then

$$\mathbf{a}_{r:s} = (a_r, a_{r+1}, \dots, a_s)^T.$$

For the stacked vector above,

$$\mathbf{b} = \mathbf{a}_{1:m}, \quad \mathbf{c} = \mathbf{a}_{m+1:m+n}, \quad \mathbf{d} = \mathbf{a}_{m+n+1:m+n+p}.$$

This notation will later be useful for selecting portions of vectors, rows of data, and groups of variables.

2.3 Indexing Conventions

In the mathematical notation used in this course, vectors are usually *1-indexed*. That means the first entry of $\mathbf{x}$ is x_1 , the second entry is x_2 , and the final entry is x_n .

Some programming languages, including Python and C/C++, are *0-indexed*. In those languages, the first entry of a stored vector or array has index 0. This difference is not a mathematical disagreement; it is a convention. When translating between mathematics and code, always check whether an index begins at 0 or 1.

2.4 Special Vectors

Several vectors occur so often that they receive special notation.

The zero vector. The *zero vector* in $\mathbb{R}^n$, written $\mathbf{0}_n$ or simply $\mathbf{0}$ when the dimension is clear, contains only zeros:

$$\mathbf{0}_n = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}.$$

The ones vector. The *ones vector* in $\mathbb{R}^n$, written $\mathbf{1}_n$ or simply $\mathbf{1}$, contains only ones:

$$\mathbf{1}_n = \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix}.$$

The ones vector is especially important later because it represents an intercept column in regression and helps express sample sums and sample means.

Standard basis vectors. For $i = 1, \dots, n$, the i th *standard basis vector* $\mathbf{e}_i \in \mathbb{R}^n$ has a 1 in position i and 0 in every other position. For example, in $\mathbb{R}^3$,

$$\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \quad \mathbf{e}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

These vectors will become important when we discuss linear combinations and bases.

2.5 Sparse and Dense Vectors

A vector is called *sparse* if many of its entries are zero. A vector that is not sparse is often called *dense*. The *sparsity pattern* of a vector is the set of positions where the entries are nonzero.

For example,

$$\begin{bmatrix} 0 \\ 0 \\ 7 \\ 0 \\ 0 \\ 2 \end{bmatrix}$$

is sparse because only two of its six entries are nonzero. We sometimes write $\text{nnz}(\mathbf{x})$ for the number of nonzero entries of $\mathbf{x}$.

Sparsity is not merely a technical detail. If a vector records the quantity of many different resources in a warehouse, sparsity means that most resource types are absent. If a vector records words in a document, sparsity means that most words in the vocabulary do not appear in that document. Recognizing sparsity can simplify computation and interpretation.

2.6 Vectors as Data and Measurements

A vector should not be thought of only as a point in physical space. That picture is useful in some settings, but the broader idea is more important:

A vector is an ordered collection of quantities that pertain to the same object, system, or concept.

Here are several common interpretations.

Vector interpretation	Example
Location	(longitude, latitude, elevation) $\in \mathbb{R}^3$
Displacement	(Δ longitude, Δ latitude, Δ elevation) $\in \mathbb{R}^3$
Color	RGB values such as (1, 0, 0) for red or (0, 1, 0) for green
Quantities	Amounts of different resources, products, or inventory classes
Time series	Values of a signal or measurement at successive times
Data record	Measurements or features describing one observation

The same mathematical object can therefore support different interpretations. In operations research and data analysis, this flexibility is essential. A vector might describe a ship's position, a unit's inventory, a time series of sensor readings, or one row of a data table.

Data Analysis bridge. In the Data Analysis textbook, a feature vector is one observation's ordered collection of input measurements. A response vector is the collection of observed outputs over many observations. Later, a residual vector will collect the leftover errors from a fitted model. All of these are instances of the same basic object: a vector in some $\mathbb{R}^n$.

2.7 Probability Vectors and Time Series

Two examples deserve special attention because they recur often in modeling.

A *probability vector* records the probabilities of a finite set of outcomes. For example, for a fair six-sided die,

$$\mathbf{w} = \left(\frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6} \right)^T.$$

For $\mathbf{w}$ to be a valid probability vector, its entries must be nonnegative and must sum to 1:

$$w_i \geq 0 \quad \text{for all } i, \quad \sum_{i=1}^n w_i = 1.$$

Probability vectors will later connect naturally to weighted averages and expectations.

A *time-series vector* records the value of some quantity at ordered time points. For example, an audio signal can be represented as a vector whose entries record acoustic pressure at equally spaced times. When a vector represents a time series, it is natural to plot the entry x_i against the time index i .

2.8 Chapter Summary

By the end of this chapter, you should be able to:

- explain what $\mathbb{R}^n$ means;
- distinguish scalars from vectors;
- identify vector entries, components, and coordinates;

- use subvector notation such as $\mathbf{a}_{r:s}$;
- recognize the zero vector, ones vector, and standard basis vectors;
- explain the difference between sparse and dense vectors;
- interpret vectors as data records, quantity records, probability vectors, or time series;
- explain why vectors in this course are not limited to physical-space arrows.

The next chapter introduces vector operations: addition, subtraction, scalar multiplication, and linear combinations.

Chapter 3

Vector Operations and Linear Combinations

The previous chapter introduced vectors as ordered collections of real numbers. We now begin to do arithmetic with them. The operations in this chapter are simple, but they are foundational: vector addition describes how quantities combine, vector subtraction describes differences, scalar multiplication describes scaling, and linear combinations describe weighted sums of vectors.

These operations are the language behind many later ideas. Residuals are differences of vectors. Centered variables are data vectors with their mean removed. Linear models are built by weighting and adding input quantities. Least squares will eventually choose the best linear combination of columns in a matrix.

3.1 Vector Addition

Two vectors can be added only when they have the same dimension. If

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

are both in $\mathbb{R}^n$, then their sum is defined entry by entry:

$$\mathbf{x} + \mathbf{y} = \begin{bmatrix} x_1 + y_1 \\ x_2 + y_2 \\ \vdots \\ x_n + y_n \end{bmatrix}.$$

For example,

$$\begin{bmatrix} 0 \\ 7 \\ 3 \end{bmatrix} + \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 9 \\ 3 \end{bmatrix}.$$

The requirement that the vectors have the same size matters. Adding a 3-vector to a 2-vector is not defined because there is no entry-by-entry pairing for all components.

Vector addition inherits the usual commutative and associative behavior of real-number addition:

$$\mathbf{x} + \mathbf{y} = \mathbf{y} + \mathbf{x}, \quad (\mathbf{x} + \mathbf{y}) + \mathbf{z} = \mathbf{x} + (\mathbf{y} + \mathbf{z}).$$

These identities hold because they hold entry by entry.

3.2 Vector Subtraction and Differences

Vector subtraction is also defined entry by entry. For $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$,

$$\mathbf{x} - \mathbf{y} = \begin{bmatrix} x_1 - y_1 \\ x_2 - y_2 \\ \vdots \\ x_n - y_n \end{bmatrix}.$$

For example,

$$\begin{bmatrix} 1 \\ 9 \end{bmatrix} - \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 8 \end{bmatrix}.$$

Subtraction is the operation of difference. If two vectors represent observed and predicted values, their difference is an error or residual vector. If two points $\mathbf{p}$ and $\mathbf{q}$ are represented using coordinates, then $\mathbf{p} - \mathbf{q}$ is the displacement from $\mathbf{q}$ to $\mathbf{p}$. If $\mathbf{p}$ is a point and $\mathbf{a}$ is a displacement, then $\mathbf{p} + \mathbf{a}$ is the point obtained by moving $\mathbf{p}$ by the displacement $\mathbf{a}$.

The same operation appears in signal processing. If $\mathbf{s}$ is a signal of interest and $\mathbf{n}$ is noise or interference, a received signal may be represented as

$$\mathbf{x} = \mathbf{s} + \mathbf{n}.$$

This is still vector addition. The interpretation changes, but the arithmetic is the same.

Centered vectors. A particularly important difference is a centered vector. If $\mathbf{x} \in \mathbb{R}^n$ has sample mean $\bar{x}$, then

$$\mathbf{x} - \bar{x}\mathbf{1}$$

is the vector of deviations from the mean. Its i th entry is $x_i - \bar{x}$. This centered-vector notation will later support variance, covariance, correlation, residual sums of squares, and degrees of freedom.

3.3 A Notational Warning: Vectors and Scalars

Some programming languages allow expressions such as

$$\begin{bmatrix} 1 \\ 7 \\ 5 \end{bmatrix} + 3 = \begin{bmatrix} 4 \\ 10 \\ 8 \end{bmatrix},$$

where the scalar is automatically added to every entry of the vector. This is convenient in code, but it is not the standard mathematical notation used in this course. In mathematical writing, we will not add a scalar directly to a vector unless a convention has been explicitly stated.

There is, however, a standard way to combine a scalar and a vector: scalar multiplication.

3.4 Scalar Multiplication

If $\alpha \in \mathbb{R}$ is a scalar and $\mathbf{x} \in \mathbb{R}^n$, then $\alpha\mathbf{x}$ is the vector obtained by multiplying every entry of $\mathbf{x}$ by α :

$$\alpha\mathbf{x} = \alpha \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} \alpha x_1 \\ \alpha x_2 \\ \vdots \\ \alpha x_n \end{bmatrix}.$$

For example,

$$(-2) \begin{bmatrix} 1 \\ 9 \\ 6 \end{bmatrix} = \begin{bmatrix} -2 \\ -18 \\ -12 \end{bmatrix}.$$

Scalar multiplication has a natural interpretation. If $\mathbf{x}$ is an audio signal, then $\beta\mathbf{x}$ is the same signal scaled in volume by the factor $|\beta|$. If $\beta < 0$, the sign of every entry is reversed as well as scaled.

The basic distributive identities are

$$(\alpha + \beta)\mathbf{x} = \alpha\mathbf{x} + \beta\mathbf{x}, \quad \alpha(\mathbf{x} + \mathbf{y}) = \alpha\mathbf{x} + \alpha\mathbf{y}.$$

These rules are again entrywise consequences of the corresponding real-number rules.

3.5 Linear Combinations

Vector addition and scalar multiplication combine into the most important operation in linear algebra: the linear combination.

Let $\mathbf{a}_1, \dots, \mathbf{a}_m$ be vectors in $\mathbb{R}^n$, and let $\beta_1, \dots, \beta_m$ be scalars. The vector

$$\beta_1\mathbf{a}_1 + \beta_2\mathbf{a}_2 + \cdots + \beta_m\mathbf{a}_m$$

is called a *linear combination* of $\mathbf{a}_1, \dots, \mathbf{a}_m$. The scalars $\beta_1, \dots, \beta_m$ are the *coefficients* of the linear combination.

The word linear means that we are only scaling vectors and adding the results. We are not multiplying the vectors together, raising them to powers, or applying nonlinear functions to their entries.

Linear combinations are the basic algebraic form behind linear models. A linear model takes measured inputs, assigns coefficients to them, and adds the weighted pieces. Later, when we write a fitted value as a matrix times a coefficient vector, we are saying that the fitted value vector is a linear combination of the columns of the matrix.

3.6 Standard Basis Representation

The standard basis vectors from Chapter 2 give the simplest example of a linear combination. In $\mathbb{R}^n$, every vector can be written as a linear combination of the standard basis vectors:

$$\mathbf{b} = b_1\mathbf{e}_1 + b_2\mathbf{e}_2 + \cdots + b_n\mathbf{e}_n.$$

The coefficients are exactly the entries of $\mathbf{b}$.

For example,

$$\begin{bmatrix} -1 \\ 3 \\ 5 \end{bmatrix} = (-1) \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} + 3 \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} + 5 \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

This example shows why the standard basis is useful: it separates a vector into coordinate directions. The first coefficient controls movement in the $\mathbf{e}_1$ direction, the second controls movement in the $\mathbf{e}_2$ direction, and so on.

3.7 Lines and Line Segments

Let $\mathbf{a}$ and $\mathbf{b}$ be two different vectors in $\mathbb{R}^n$. For a scalar θ , define

$$\mathbf{c}(\theta) = (1 - \theta)\mathbf{a} + \theta\mathbf{b}.$$

As θ varies over all real numbers, $\mathbf{c}(\theta)$ traces the line through $\mathbf{a}$ and $\mathbf{b}$. When

$$0 \leq \theta \leq 1,$$

$\mathbf{c}(\theta)$ lies on the line segment connecting $\mathbf{a}$ and $\mathbf{b}$.

This formula has a simple interpretation. When $\theta = 0$,

$$\mathbf{c}(0) = \mathbf{a}.$$

When $\theta = 1$,

$$\mathbf{c}(1) = \mathbf{b}.$$

When $\theta = 1/2$,

$$\mathbf{c}(1/2) = \frac{1}{2}\mathbf{a} + \frac{1}{2}\mathbf{b},$$

which is the midpoint between $\mathbf{a}$ and $\mathbf{b}$. Values of θ between 0 and 1 interpolate between the two endpoints; values outside this interval extend the line beyond one endpoint.

The expression $(1 - \theta)\mathbf{a} + \theta\mathbf{b}$ is sometimes called an affine combination because its coefficients sum to 1:

$$(1 - \theta) + \theta = 1.$$

Affine combinations will matter later when we distinguish ordinary linear combinations from shifted geometric descriptions such as lines, planes, and intercept-containing models.

3.8 Data Analysis Bridge

The operations in this chapter appear constantly in data analysis:

- residual vectors are differences, such as $\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}}$;
- centered variables are differences, such as $\mathbf{x} - \bar{x}\mathbf{1}$;
- scaled variables use scalar multiplication;

- linear predictors are weighted sums of input quantities;
- fitted response vectors are linear combinations of design-matrix columns.

The same small set of vector operations therefore supports prediction error, regression, correlation, least squares, and model diagnostics.

3.9 Chapter Summary

By the end of this chapter, you should be able to:

- add and subtract vectors of the same dimension;
- explain why vectors of different dimensions cannot be added directly;
- interpret subtraction as a difference or displacement;
- distinguish mathematical vector notation from programming broadcast conventions;
- multiply a vector by a scalar;
- use the distributive laws for scalar multiplication;
- define a linear combination and identify its coefficients;
- write a vector as a linear combination of standard basis vectors;
- describe the line segment $(1 - \theta)\mathbf{a} + \theta\mathbf{b}$ for $0 \leq \theta \leq 1$.

The next chapter returns to set theory and quantifiers. That material complements the logic chapter and provides language for membership, subsets, complements, unions, intersections, and indexed collections.

Chapter 4

Sets, Set Operations, and Quantifiers

The previous chapters introduced vectors and vector operations. We now return to set theory, which gives precise language for membership, exclusion, filtering, and collections of objects. In operations research and data analysis, set notation appears whenever we describe feasible choices, valid values, selected observations, selected variables, or assumptions that must hold over a collection.

The main idea is simple: a set is a collection of objects, and a membership statement such as $x \in S$ is either true or false. We can therefore combine set notation with the logical connectives from Chapter 1.

4.1 Sets and Membership

If S is a set, then

$$x \in S$$

means that x is an element of S , while

$$x \notin S$$

means that x is not an element of S . A set may be described by listing its elements, such as

$$A = \{2, 4, 6, 8, 10\},$$

or by giving a membership rule, such as

$$S = \{x \in \mathbb{R} : 0 \leq x \leq 1\}.$$

Here the universe is $\mathbb{R}$, and the rule says that a real number belongs to S exactly when it lies between 0 and 1, inclusive.

The complement of S , written S^c , is the set of objects in the relevant universe that do not belong to S :

$$S^c = \{x : x \notin S\}.$$

For the interval above,

$$S^c = \{x \in \mathbb{R} : x < 0 \text{ or } x > 1\}.$$

This follows by negating the membership condition:

$$x \notin S \iff \neg(x \geq 0 \text{ and } x \leq 1) \iff x < 0 \text{ or } x > 1.$$

4.2 Set Operations

Let R and S be sets. Their *union* is

$$R \cup S = \{x : x \in R \text{ or } x \in S\},$$

and their *intersection* is

$$R \cap S = \{x : x \in R \text{ and } x \in S\}.$$

The “or” in the union is inclusive: if x belongs to both sets, then $x \in R \cup S$.

Two other operations are especially useful. The *set difference* is

$$R \setminus S = \{x : x \in R \text{ and } x \notin S\} = R \cap S^c.$$

It keeps the part of R that is not in S . The *symmetric difference* is

$$R \triangle S = (R \setminus S) \cup (S \setminus R) = (R \cap S^c) \cup (S \cap R^c).$$

It keeps the objects that belong to exactly one of the two sets.

A useful way to remember these operations is through membership equivalences:

$$\begin{aligned} x \in A \cup B &\iff (x \in A) \vee (x \in B), \\ x \in A \cap B &\iff (x \in A) \wedge (x \in B), \\ x \in A^c &\iff \neg(x \in A), \\ x \in A \setminus B &\iff (x \in A) \wedge \neg(x \in B), \\ x \in A \triangle B &\iff ((x \in A) \wedge \neg(x \in B)) \vee ((x \in B) \wedge \neg(x \in A)). \end{aligned}$$

4.3 De Morgan’s Laws for Sets

The De Morgan laws from logic have exact set versions:

$$(A \cap B)^c = A^c \cup B^c, \quad (A \cup B)^c = A^c \cap B^c.$$

The first law says that not being in both A and B is the same as being outside A or outside B .

The second says that not being in either A or B is the same as being outside both.

These laws are just logical equivalences applied to membership statements. For example,

$$\begin{aligned} x \in (A \cap B)^c &\iff \neg(x \in A \cap B) \\ &\iff \neg((x \in A) \wedge (x \in B)) \\ &\iff \neg(x \in A) \vee \neg(x \in B) \\ &\iff x \in A^c \cup B^c. \end{aligned}$$

In data-cleaning language, the negation of “valid height and valid weight” is “invalid height or invalid weight.”

4.4 Subsets, Equality, and Special Sets

A set B is a *subset* of A , written $B \subseteq A$, if every element of B is also an element of A :

$$B \subseteq A \iff \forall x (x \in B \Rightarrow x \in A).$$

Two sets are equal when they contain exactly the same elements:

$$A = B \iff \forall x (x \in A \Leftrightarrow x \in B).$$

A common way to prove $A = B$ is to show both $A \subseteq B$ and $B \subseteq A$.

Some special sets occur frequently:

- $\emptyset$ is the empty set, containing no elements.
- A singleton is a set with exactly one element, such as $\{5\}$.
- The power set $\mathcal{P}(A)$, also written 2^A , is the set of all subsets of A .

For example, if $A = \{1, 2\}$, then

$$\mathcal{P}(A) = \{\emptyset, \{1\}, \{2\}, \{1, 2\}\}.$$

4.5 Families of Sets and a Finite Example

A collection of sets is called a *family of sets*. If $\mathcal{F} = \{A, B, C\}$, then

$$\bigcap_{S \in \mathcal{F}} S \quad \text{and} \quad \bigcup_{S \in \mathcal{F}} S$$

mean the intersection and union over all sets in the family. Membership is defined by

$$x \in \bigcap_{S \in \mathcal{F}} S \iff x \in S \text{ for every } S \in \mathcal{F},$$

and

$$x \in \bigcup_{S \in \mathcal{F}} S \iff x \in S \text{ for at least one } S \in \mathcal{F}.$$

As a finite example, let

$$U = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\},$$

$$A = \{2, 4, 6, 8, 10\}, \quad B = \{3, 6, 9\}, \quad C = \{4, 8\}.$$

Then

$$A \cup C = A, \quad A \cap C = C, \quad C \subseteq A,$$

and

$$A \setminus B = \{2, 4, 8, 10\}.$$

We can also sum over sets:

$$\sum_{x \in A} x = 30, \quad \sum_{x \in A \cap B} x = 6, \quad \sum_{x \in A \setminus B} x = 24.$$

This notation appears whenever a calculation is restricted to selected elements.

4.6 Common Number Systems

The notes use several standard number systems:

$$\begin{aligned}\mathbb{N} &= \{1, 2, 3, \dots\}, \\ \mathbb{Z} &= \{\dots, -2, -1, 0, 1, 2, \dots\}, \\ \mathbb{Q} &= \left\{\frac{a}{b} : a, b \in \mathbb{Z}, b \neq 0\right\}, \\ \mathbb{R} &= \text{the real numbers}, \\ \mathbb{C} &= \text{the complex numbers}.\end{aligned}$$

The set difference $\mathbb{R} \setminus \mathbb{Q}$ is the set of irrational real numbers.

4.7 Quantified Statements

Quantifiers allow us to write statements about many objects at once. The universal quantifier $\forall$ means “for all,” and the existential quantifier $\exists$ means “there exists.” For example, “every multiple of 6 is divisible by 3” can be written as

$$\forall x \in \mathbb{Z}, \quad 6 \mid x \Rightarrow 3 \mid x.$$

Here $6 \mid x$ means “6 divides x .”

Quantifier order matters. The statement

$$\exists x \in \mathbb{Z} \text{ such that } \forall y \in \mathbb{R}, xy = -1$$

is false because no single integer x can make $xy = -1$ for every real number y . The statement

$$\forall y \in \mathbb{R} \exists x \in \mathbb{Z} \text{ such that } xy = -1$$

is also false; for example, if $y = 2$, the required value would be $x = -1/2 \notin \mathbb{Z}$.

The following examples illustrate the same point:

- $\exists y \in \mathbb{Z}$ such that $\forall x \in \mathbb{Z}, xy \geq 0$ is true; choose $y = 0$.
- $\forall x \in \mathbb{Z}, \exists y \in \mathbb{Z}$ such that $xy > 0$ is false; $x = 0$ is a counterexample.
- If the intended statement is “for every nonzero integer x , there exists an integer y such that $xy > 0$,” then it is true; choose $y = x$.

4.8 Negating Quantified Statements

The two basic rules for negating quantified statements are

$$\neg(\exists x \text{ such that } P(x)) \iff \forall x, \neg P(x),$$

and

$$\neg(\forall x, P(x)) \iff \exists x \text{ such that } \neg P(x).$$

In words: to disprove an existential statement, show that no object works; to disprove a universal statement, give one counterexample.

For example, the statement

$$\forall x \in \mathbb{Z}, \quad x > 0$$

is false because its negation is

$$\exists x \in \mathbb{Z} \text{ such that } x \leq 0,$$

and $x = 0$ is a counterexample.

4.9 Data Analysis Bridge

Set notation is one of the cleanest ways to describe data-cleaning decisions. If I is the set of row indices in a data table and $B \subseteq I$ is the set of rows with invalid weights, then $I \setminus B$ is the set of rows retained after removing invalid cases. If J is the set of variable names and $K \subseteq J$ is the set of variables selected for a model, then K is a subset of the available columns.

Quantifiers make cleaning rules defensible. Instead of saying “remove bad rows,” an analyst should be able to state a predicate such as “remove row i if `weight $i$` is missing or `weight $i$  ≤ 0`.” That predicate identifies a set of rows by a rule, and the cleaned data set is obtained by set difference.

4.10 Chapter Summary

By the end of this chapter, you should be able to:

- use $x \in S$ and $x \notin S$ to describe membership and nonmembership;
- define union, intersection, complement, set difference, and symmetric difference;
- translate set operations into logical statements about membership;
- use De Morgan’s laws for complements of unions and intersections;
- recognize subsets, set equality, the empty set, singletons, power sets, and families of sets;
- use $\forall$ and $\exists$ in quantified statements;
- negate quantified statements correctly;
- describe filtering, subsetting, and cleaning rules with set notation.

The next chapter returns to geometry. It introduces dot products, norms, distance, angles, and orthogonality. These tools measure lengths and compare directions, preparing the way for projection and least squares.

Chapter 5

Dot Products, Norms, Distance, Angles, and Orthogonality

The previous chapters treated vectors as ordered collections of quantities and described how to add, subtract, scale, and combine them. Those operations produce new vectors. This chapter introduces operations that use vectors to produce scalars: dot products, lengths, distances, and angles.

These ideas are central because they connect algebra, geometry, computation, and later least-squares reasoning. A dot product is a weighted sum. A norm is a length. A distance compares two vectors. An angle measures alignment. Orthogonality is the special case of zero alignment.

5.1 The Inner Product

Let $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$. The *inner product*, also called the *dot product*, of $\mathbf{a}$ and $\mathbf{b}$ is the scalar

$$\mathbf{a}^T \mathbf{b} = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n = \sum_{i=1}^n a_i b_i.$$

Some references write this as $\langle \mathbf{a}, \mathbf{b} \rangle$, $(\mathbf{a} \mid \mathbf{b})$, or $\mathbf{a} \cdot \mathbf{b}$. In this text, we primarily use $\mathbf{a}^T \mathbf{b}$, because that notation connects naturally to matrix algebra later.

The two vectors must have the same length. The operation multiplies matching entries and then adds the products. For example,

$$\begin{bmatrix} -1 \\ 2 \\ 2 \end{bmatrix}^T \begin{bmatrix} 1 \\ 0 \\ -3 \end{bmatrix} = (-1)(1) + (2)(0) + (2)(-3) = -7.$$

The result is not a vector. It is a single number.

5.2 Properties of the Inner Product

The inner product has three basic algebraic properties that we will use repeatedly:

$$\begin{aligned} \mathbf{a}^T \mathbf{b} &= \mathbf{b}^T \mathbf{a}, & \text{commutativity,} \\ (\gamma \mathbf{a})^T \mathbf{b} &= \gamma(\mathbf{a}^T \mathbf{b}), & \text{compatibility with scalar multiplication,} \\ (\mathbf{a} + \mathbf{b})^T \mathbf{c} &= \mathbf{a}^T \mathbf{c} + \mathbf{b}^T \mathbf{c}, & \text{distributivity over addition.} \end{aligned}$$

Together, these say that the inner product behaves like a linear measuring device. It converts a vector into a scalar by weighting and adding entries, and it respects vector addition and scalar multiplication.

5.3 Inner Products as Selection, Sums, Means, and Weighted Sums

The inner product is useful because many common calculations are special cases of weighted sums.

Let $\mathbf{e}_i \in \mathbb{R}^n$ be the i th standard basis vector. Then

$$\mathbf{e}_i^T \mathbf{a} = a_i.$$

Thus $\mathbf{e}_i^T$ acts as a selection operator: it selects the i th entry of $\mathbf{a}$.

Let $\mathbf{1} \in \mathbb{R}^n$ be the ones vector. Then

$$\mathbf{1}^T \mathbf{a} = \sum_{i=1}^n a_i.$$

The sample mean of the entries of $\mathbf{a}$ can therefore be written as

$$\frac{1}{n} \mathbf{1}^T \mathbf{a} = \frac{1}{n} \sum_{i=1}^n a_i.$$

Finally,

$$\mathbf{a}^T \mathbf{a} = \sum_{i=1}^n a_i^2,$$

which is the sum of squared entries. This expression becomes the foundation for squared length, residual sum of squares, and least squares.

Example: prices and quantities. Suppose $\mathbf{p} \in \mathbb{R}^n$ is a vector of prices for n goods, and $\mathbf{q} \in \mathbb{R}^n$ is a vector of quantities to purchase. Then

$$\mathbf{p}^T \mathbf{q} = p_1 q_1 + \cdots + p_n q_n$$

is the total cost of the goods. The price vector supplies the weights; the quantity vector supplies the amounts being weighted.

This same pattern appears in probability. If $\mathbf{p}$ is a probability vector and $\mathbf{x}$ is a vector of possible outcome values, then $\mathbf{p}^T \mathbf{x}$ is the expected value of the outcome. It is a weighted average written as an inner product.

5.4 Computational Cost of Vector Operations

Vector operations are not only algebraic; they are also computational. Real numbers are usually stored in a computer using floating-point format. In double precision, each number uses 64 bits, or 8 bytes. Thus, a vector with n double-precision entries requires about $8n$ bytes of memory.

For example, 16 GB of memory is approximately 16 billion bytes. At 8 bytes per number, this is enough storage for about 2 billion double-precision numbers, ignoring overhead.

The time required for a vector calculation is often estimated by counting floating-point operations, or *flops*. The exact runtime depends on hardware, memory access, and implementation details, but a flop count gives a useful first approximation.

Common vector operations have the following approximate costs:

scalar-vector multiplication	$\alpha \mathbf{x}$	n multiplications,
vector addition	$\mathbf{x} + \mathbf{y}$	n additions,
inner product	$\mathbf{x}^T \mathbf{y}$	n multiplications and $n - 1$ additions.

Thus an inner product requires $2n - 1 \approx 2n$ flops. For small vectors this distinction is not important. For very large vectors or repeated computations, it becomes part of algorithmic reasoning.

5.5 The Euclidean Norm

The *Euclidean norm* of $\mathbf{x} \in \mathbb{R}^n$ is

$$\|\mathbf{x}\| = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}.$$

Using the inner product, this can be written compactly as

$$\|\mathbf{x}\| = \sqrt{\mathbf{x}^T \mathbf{x}}.$$

The Euclidean norm is also called length, magnitude, or simply norm when the context is clear. A vector with norm 1 is called a *unit vector*; we also say the vector has been normalized.

For example, if

$$\mathbf{x} = \begin{bmatrix} 3 \\ 4 \end{bmatrix},$$

then

$$\|\mathbf{x}\| = \sqrt{3^2 + 4^2} = 5.$$

This is the familiar Pythagorean length formula, now written for vectors.

5.6 Norm Properties

The Euclidean norm satisfies three important properties:

$$\begin{aligned} \|\beta \mathbf{x}\| &= |\beta| \|\mathbf{x}\|, & \text{positive homogeneity,} \\ \|\mathbf{x} + \mathbf{y}\| &\leq \|\mathbf{x}\| + \|\mathbf{y}\|, & \text{subadditivity or triangle inequality,} \\ \|\mathbf{x}\| &\geq 0, \quad \|\mathbf{x}\| = 0 \Rightarrow \mathbf{x} = \mathbf{0}, & \text{positive definiteness.} \end{aligned}$$

Any function satisfying these properties is called a norm. In this course, unless otherwise stated, $\|\cdot\|$ means the Euclidean norm.

These properties match geometric intuition. Scaling a vector by β scales its length by $|\beta|$. The length of a sum is no greater than the sum of lengths. Only the zero vector has zero length.

5.7 Distance Between Vectors

The distance between two vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$ is defined as the length of their difference:

$$\text{dist}(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\| = \sqrt{(a_1 - b_1)^2 + \cdots + (a_n - b_n)^2}.$$

Thus distance is not a new operation separate from subtraction and norm. First subtract the vectors, then measure the length of the resulting displacement.

This is why squared prediction error and residual error later become geometric ideas. If $\mathbf{y}$ is an observed response vector and $\hat{\mathbf{y}}$ is a fitted response vector, then $\|\mathbf{y} - \hat{\mathbf{y}}\|$ is the distance between what was observed and what the model fitted.

5.8 Angles and Orthogonality

For nonzero vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$, the angle θ between them is defined by

$$\theta = \cos^{-1} \left(\frac{\mathbf{a}^T \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|} \right),$$

where $0 \leq \theta \leq \pi$. Equivalently,

$$\mathbf{a}^T \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos(\theta).$$

This formula shows that the dot product measures both length and alignment.

Some special cases are especially important:

$$\begin{aligned} \theta = 0 &\implies \mathbf{a}^T \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\|, \\ \theta = \frac{\pi}{2} &\implies \mathbf{a}^T \mathbf{b} = 0. \end{aligned}$$

When $\mathbf{a}^T \mathbf{b} = 0$, the vectors are called *orthogonal*. In two or three dimensions, this means a right angle. In higher dimensions, it is the correct generalization of perpendicularity.

The angle formula is defined only for nonzero vectors because the denominator contains $\|\mathbf{a}\| \|\mathbf{b}\|$. The zero vector is orthogonal to every vector in the algebraic sense $\mathbf{0}^T \mathbf{b} = 0$, but it does not have a meaningful angle with other vectors.

5.9 Example: Earth Geometry

Distance depends on the geometry being used. As a useful example, approximate Earth as a sphere with radius

$$R = 6367.5 \text{ km.}$$

Using a standard latitude-longitude convention, a point with latitude θ and longitude λ can be represented by the three-dimensional vector

$$\mathbf{r}(\theta, \lambda) = R \begin{bmatrix} \cos \theta \cos \lambda \\ \cos \theta \sin \lambda \\ \sin \theta \end{bmatrix}.$$

If $\mathbf{a}$ and $\mathbf{b}$ are two such position vectors, then $\|\mathbf{a} - \mathbf{b}\|$ is the straight-line distance through Earth, also called the chord distance. The distance along the surface is found from the central angle between $\mathbf{a}$ and $\mathbf{b}$:

$$\alpha = \cos^{-1} \left(\frac{\mathbf{a}^T \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|} \right), \quad \text{surface distance} = R\alpha.$$

For the Beijing and Palo Alto coordinates in the source notes, the through-Earth distance is approximately

$$8674.78 \text{ km},$$

and the surface distance is approximately

$$9543.20 \text{ km}.$$

The example illustrates a general point: norms and angles are not abstract decorations. They let us turn geometry into computable formulas.

5.10 Data Analysis Bridge

This chapter supplies several mathematical tools that the Data Analysis textbook uses repeatedly:

- Dot products write weighted sums such as $\beta^T \mathbf{x}$, expected values, sample means, and sums of products.
- The expression $\mathbf{a}^T \mathbf{a}$ writes sums of squared entries compactly.
- Euclidean norms and distances describe residual length, MSE, RMSE, RSS, and TSS.
- Angles describe alignment between centered data vectors, which is the geometric meaning behind correlation.
- Orthogonality drives projections, normal equations, and residual geometry.

Later regression chapters use these ideas to express least squares as choosing the fitted vector closest to the observed response vector.

5.11 Chapter Summary

By the end of this chapter, you should be able to:

- compute the dot product $\mathbf{a}^T \mathbf{b}$;
- interpret dot products as weighted sums;

- use $\mathbf{1}^T \mathbf{a}$, $\frac{1}{n} \mathbf{1}^T \mathbf{a}$, and $\mathbf{a}^T \mathbf{a}$;
- define and compute the Euclidean norm $\|\mathbf{x}\|$;
- define distance as $\|\mathbf{a} - \mathbf{b}\|$;
- relate dot products to angles;
- recognize orthogonality through the condition $\mathbf{a}^T \mathbf{b} = 0$;
- explain why these ideas matter for sums of squares, correlation, and least squares.

Chapter 6

Linear Independence, Span, Bases, and Dimension

The previous chapters treated vectors as ordered collections of quantities and described how to add, subtract, scale, combine, compare, and measure them. We now ask a structural question: when does a collection of vectors contain genuinely different directions, and when is one vector redundant because it can be built from the others?

This chapter develops four connected ideas. The *span* of a set of vectors is the collection of all vectors that can be built from them by linear combinations. *Linear independence* tells us whether any vector in the collection is redundant. A *basis* is a nonredundant set of directions that can represent every vector in a space. The *dimension* of a space counts how many independent directions are needed to describe it.

These ideas will later support matrix rank, invertibility, least squares, degrees of freedom, and identifiability. For now, we stay with vectors and linear combinations.

6.1 Span: All Vectors Built from a Collection

Let $\mathbf{a}_1, \dots, \mathbf{a}_k \in \mathbb{R}^n$. The *span* of these vectors is the set of all linear combinations of them:

$$\text{span}\{\mathbf{a}_1, \dots, \mathbf{a}_k\} = \{\beta_1 \mathbf{a}_1 + \dots + \beta_k \mathbf{a}_k : \beta_1, \dots, \beta_k \in \mathbb{R}\}.$$

In words, the span is everything we can make by scaling and adding the listed vectors.

If $\mathbf{a}_1$ is a single nonzero vector, then

$$\text{span}\{\mathbf{a}_1\} = \{\beta \mathbf{a}_1 : \beta \in \mathbb{R}\},$$

which is the line through the origin in the direction of $\mathbf{a}_1$. If two nonzero vectors in $\mathbb{R}^3$ point in genuinely different directions, their span is a plane through the origin. If three vectors in $\mathbb{R}^3$ point in enough different directions, their span may be all of $\mathbb{R}^3$.

The span language makes linear combinations geometric. A set of vectors supplies directions; the span is the collection of all locations reachable by moving in those directions with real-valued coefficients.

6.2 Linear Dependence

A set of vectors $\{\mathbf{a}_1, \dots, \mathbf{a}_k\} \subset \mathbb{R}^n$ is *linearly dependent* if there exist scalars $\beta_1, \dots, \beta_k$, not all zero, such that

$$\beta_1 \mathbf{a}_1 + \beta_2 \mathbf{a}_2 + \dots + \beta_k \mathbf{a}_k = \mathbf{0}.$$

The phrase “not all zero” is essential. We can always make the zero vector by choosing all coefficients equal to zero. Linear dependence means there is a *nontrivial* way to make the zero vector.

For two vectors, suppose

$$\beta_1 \mathbf{a}_1 + \beta_2 \mathbf{a}_2 = \mathbf{0}$$

with $\beta_2 \neq 0$. Then

$$\mathbf{a}_2 = -\frac{\beta_1}{\beta_2} \mathbf{a}_1.$$

Thus $\mathbf{a}_2$ is just a scalar multiple of $\mathbf{a}_1$. The two vectors do not provide two different directions; one is redundant.

The same idea holds for more than two vectors. If a collection is linearly dependent, then at least one vector in the collection can be written as a linear combination of the others. To see why, suppose

$$\beta_1 \mathbf{a}_1 + \dots + \beta_k \mathbf{a}_k = \mathbf{0}$$

for coefficients not all zero. Choose an index j with $\beta_j \neq 0$. Then

$$\mathbf{a}_j = -\frac{1}{\beta_j} \sum_{i \neq j} \beta_i \mathbf{a}_i.$$

So $\mathbf{a}_j$ can be built from the remaining vectors.

A collection that contains the zero vector is automatically linearly dependent. If $\mathbf{a}_j = \mathbf{0}$, choose $\beta_j = 1$ and all other coefficients equal to zero. That is a nontrivial linear combination that equals $\mathbf{0}$.

6.3 Linear Independence

A set of vectors $\mathbf{a}_1, \dots, \mathbf{a}_k \in \mathbb{R}^n$ is *linearly independent* if it is not linearly dependent. Equivalently,

$$\beta_1 \mathbf{a}_1 + \dots + \beta_k \mathbf{a}_k = \mathbf{0} \implies \beta_1 = \beta_2 = \dots = \beta_k = 0.$$

In words, the only way to combine the vectors to get the zero vector is the trivial way: use all-zero coefficients.

Linear independence means that no vector in the set is unnecessary. Each vector contributes a direction that cannot be built from the others.

Example in $\mathbb{R}^2$. Consider

$$\mathbf{a}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \mathbf{a}_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

If

$$\beta_1 \mathbf{a}_1 + \beta_2 \mathbf{a}_2 = \mathbf{0},$$

then

$$\beta_1 \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \beta_2 \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} \beta_1 + \beta_2 \\ \beta_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

The second coordinate gives $\beta_2 = 0$. Then the first coordinate gives $\beta_1 = 0$. Therefore the two vectors are linearly independent.

6.4 How Many Independent Vectors Can There Be?

A linearly independent set of n -vectors can have at most n vectors. In $\mathbb{R}^2$, we cannot have three linearly independent vectors. In $\mathbb{R}^3$, we cannot have four linearly independent vectors. More generally, no more than n independent directions fit inside $\mathbb{R}^n$.

One way to see this is to examine the equation

$$\beta_1 \mathbf{a}_1 + \cdots + \beta_k \mathbf{a}_k = \mathbf{0}.$$

This is a system of n scalar equations, one for each coordinate, in k unknown coefficients $\beta_1, \dots, \beta_k$. If $k > n$, then there are more unknown coefficients than equations. Such a homogeneous system has a nontrivial solution, so the vectors are linearly dependent.

This fact gives the first meaning of *dimension*: $\mathbb{R}^n$ has n independent directions. The number n is not just the number of entries in a vector; it is the maximum number of mutually independent directions available in the space.

6.5 Bases

A *basis* for $\mathbb{R}^n$ is a collection of n linearly independent vectors in $\mathbb{R}^n$. Equivalently, it is a set of vectors that is both:

1. linearly independent, so none of the vectors is redundant; and
2. spanning, so every vector in $\mathbb{R}^n$ can be written as a linear combination of them.

Suppose $\mathbf{a}_1, \dots, \mathbf{a}_n$ form a basis for $\mathbb{R}^n$. Then every vector $\mathbf{b} \in \mathbb{R}^n$ can be written in the form

$$\mathbf{b} = c_1 \mathbf{a}_1 + c_2 \mathbf{a}_2 + \cdots + c_n \mathbf{a}_n$$

for some scalars $c_1, \dots, c_n$.

The source notes justify this using the maximum-size fact. The set

$$\{\mathbf{a}_1, \dots, \mathbf{a}_n, \mathbf{b}\}$$

contains $n+1$ vectors in $\mathbb{R}^n$, so it must be linearly dependent. Therefore there are coefficients, not all zero, such that

$$\beta_1 \mathbf{a}_1 + \cdots + \beta_n \mathbf{a}_n + \beta_{n+1} \mathbf{b} = \mathbf{0}.$$

The coefficient β_{n+1} cannot be zero; if it were zero, then the basis vectors themselves would be linearly dependent. Thus $\beta_{n+1} \neq 0$, and we may solve for $\mathbf{b}$:

$$\mathbf{b} = -\frac{\beta_1}{\beta_{n+1}} \mathbf{a}_1 - \cdots - \frac{\beta_n}{\beta_{n+1}} \mathbf{a}_n.$$

So every vector in $\mathbb{R}^n$ is in the span of the basis vectors.

6.6 Coordinates in a Basis Are Unique

A basis does more than merely represent every vector. It represents every vector *uniquely*.

Suppose $\mathbf{a}_1, \dots, \mathbf{a}_n$ form a basis and suppose that $\mathbf{b} \in \mathbb{R}^n$ has two representations:

$$\mathbf{b} = c_1\mathbf{a}_1 + \cdots + c_n\mathbf{a}_n$$

and

$$\mathbf{b} = d_1\mathbf{a}_1 + \cdots + d_n\mathbf{a}_n.$$

Subtract the second equation from the first:

$$\mathbf{0} = (c_1 - d_1)\mathbf{a}_1 + \cdots + (c_n - d_n)\mathbf{a}_n.$$

Because the basis vectors are linearly independent,

$$c_1 - d_1 = \cdots = c_n - d_n = 0.$$

Therefore $c_i = d_i$ for every i . The coordinates of $\mathbf{b}$ in that basis are unique.

This is the reason bases are useful. Once a basis has been chosen, a vector can be described by its coordinate weights relative to that basis.

6.7 The Standard Basis

The standard basis for $\mathbb{R}^n$ is

$$\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, \quad \mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix}, \quad \dots, \quad \mathbf{e}_n = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{bmatrix}.$$

Each $\mathbf{e}_i$ has a 1 in the i th entry and 0 everywhere else. For any vector

$$\mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix} \in \mathbb{R}^n,$$

we have

$$\mathbf{b} = b_1\mathbf{e}_1 + b_2\mathbf{e}_2 + \cdots + b_n\mathbf{e}_n.$$

Thus the standard basis expresses the usual coordinates of $\mathbf{b}$.

Other bases are possible. For example, the two vectors

$$\begin{bmatrix} 1.2 \\ -2.6 \end{bmatrix}, \quad \begin{bmatrix} -0.3 \\ -3.7 \end{bmatrix}$$

form a basis for $\mathbb{R}^2$. They point in different directions, so neither one is a scalar multiple of the other. Therefore they provide two independent directions in a two-dimensional space.

6.8 Orthonormal Bases

The standard basis is special because its vectors are both orthogonal and normal. For the standard basis vectors,

$$\mathbf{e}_i^T \mathbf{e}_j = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases}$$

A basis whose vectors are mutually orthogonal and each have norm 1 is called an *orthonormal basis*. Equivalently, for an orthonormal basis $\mathbf{q}_1, \dots, \mathbf{q}_n$,

$$\mathbf{q}_i^T \mathbf{q}_j = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases}$$

For an orthonormal basis, dot products recover coordinates directly. These bases return later with Gram-Schmidt and QR factorization. This chapter only introduces the idea; the construction is deferred.

6.9 Data Analysis Bridge

The Data Analysis textbook uses the ideas in this chapter in several places:

- A model becomes more flexible when it has more independent directions available for fitting patterns.
- Degrees of freedom count independent dimensions, not just the number of formulas on a page.
- Regression design columns must provide enough independent information for coefficients to be uniquely identified.
- Multicollinearity is a practical warning that some design columns are nearly, or exactly, redundant.

Later chapters reuse this vocabulary for matrix rank, column space, null space, and least-squares identifiability. Here the key point is simpler: independent vectors add directions, dependent vectors repeat directions, bases give unique coordinates, and dimension counts directions.

6.10 Chapter Summary

By the end of this chapter, you should be able to:

- define the span of a set of vectors;
- distinguish linear dependence from linear independence;
- explain why a dependent collection contains a redundant vector;
- state why no more than n linearly independent vectors can exist in $\mathbb{R}^n$;
- define a basis for $\mathbb{R}^n$;

- explain why basis coordinates are unique; and
- recognize the standard basis and the idea of an orthonormal basis.

Chapter 7

Matrices and Matrix-Vector Multiplication

The previous chapters treated vectors as ordered collections of quantities and described how to add, subtract, scale, combine, compare, and measure them. We now move from a single vector to an array of vectors. A *matrix* is a rectangular table of numbers, and it is one of the main tools for organizing systems of equations, data tables, and linear transformations.

In operations research and data analysis, matrices usually play two roles at once. First, they organize information: rows may represent observations, constraints, or time steps, while columns may represent variables, resources, or directions. Second, they act on vectors: multiplying a matrix by a vector produces a new vector. This chapter introduces matrix notation and the two most important interpretations of matrix-vector multiplication.

We keep the focus narrow. Rank, inverses, subspaces, QR factorization, and least squares all depend on the ideas in this chapter, but they are developed later. Here the goal is to become fluent with dimensions, entries, rows, columns, transposes, special matrices, and matrix-vector products.

7.1 Matrices as Rectangular Arrays

A *matrix* is a rectangular array of numbers. For example,

$$B = \begin{bmatrix} 0 & 1 & 7.3 & 0.1 \\ 1.3 & 4 & -0.1 & 0 \\ 4.1 & -1 & 0 & 1.7 \end{bmatrix} \in \mathbb{R}^{3 \times 4}.$$

The notation $B \in \mathbb{R}^{3 \times 4}$ means that B has 3 rows and 4 columns. More generally, a matrix $A \in \mathbb{R}^{m \times n}$ has m rows and n columns.

The entries of a matrix are indexed by row and column. We write A_{ij} for the entry in row i and column j . For the matrix B above,

$$B_{22} = 4,$$

using the mathematical convention that indices begin at 1.

A matrix may also be described by its shape:

- a *square matrix* has the same number of rows and columns;

- a *tall matrix* has more rows than columns;
- a *wide matrix* has more columns than rows.

The shape of a matrix is not just bookkeeping. It tells us whether operations are defined. Many matrix mistakes are dimension mistakes: the entries may be reasonable, but the shapes do not match the operation being attempted.

7.2 Rows, Columns, Blocks, and Submatrices

A matrix can be viewed by entries, by rows, by columns, or by blocks. If $A \in \mathbb{R}^{m \times n}$, then its i th row is often written as $A_{i,:}$, and its j th column is written as $A_{:,j}$. The colon notation means “take all entries in that direction.”

For example, the matrix

$$B = \begin{bmatrix} 0 & 1 & 7.3 & 0.1 \\ 1.3 & 4 & -0.1 & 0 \\ 4.1 & -1 & 0 & 1.7 \end{bmatrix}$$

has third column

$$B_{:,3} = \begin{bmatrix} 7.3 \\ -0.1 \\ 0 \end{bmatrix},$$

and second row

$$B_{2,:} = [1.3 \quad 4 \quad -0.1 \quad 0].$$

We can also extract submatrices. The notation $B_{2:3,3:4}$ means rows 2 through 3 and columns 3 through 4:

$$B_{2:3,3:4} = \begin{bmatrix} -0.1 & 0 \\ 0 & 1.7 \end{bmatrix}.$$

This is the matrix version of subvector or slice notation.

Matrices are sometimes written in block form. For instance,

$$A = \begin{bmatrix} B & C \\ D & E \end{bmatrix}$$

means that A has been built from smaller matrices. The block sizes must be compatible: blocks in the same block row must have the same number of rows, and blocks in the same block column must have the same number of columns.

Block notation makes many large matrix expressions easier to understand as combinations of smaller matrices.

7.3 Special Matrices

Several special matrices appear often enough that they deserve names.

The *zero matrix* is a matrix whose entries are all zero. We write it as 0 when the dimensions are clear, or as $0_{m \times n}$ when we need to specify the shape.

The *identity matrix* is a square matrix that leaves vectors unchanged under multiplication. It is written $I_n \in \mathbb{R}^{n \times n}$. Its entries are

$$(I_n)_{ij} = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases}$$

For example,

$$I_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

The identity matrix is the matrix analogue of the number 1. It is the multiplicative “do nothing” matrix.

A matrix is *sparse* if many of its entries are zero. The notation $\text{nnz}(A)$ counts the number of nonzero entries in A . If $A \in \mathbb{R}^{m \times n}$, then

$$\frac{\text{nnz}(A)}{mn}$$

is the fraction of entries that are nonzero. Sparse matrices are important in computation because it is often wasteful to store and operate on large numbers of zeros.

Triangular matrices. A square matrix is *upper triangular* if all entries below the main diagonal are zero:

$$A_{ij} = 0 \quad \text{whenever } i > j.$$

For example,

$$\begin{bmatrix} 2 & 3 & 9 \\ 0 & 4 & 8 \\ 0 & 0 & 1 \end{bmatrix}$$

is upper triangular. A square matrix is *lower triangular* if all entries above the main diagonal are zero:

$$A_{ij} = 0 \quad \text{whenever } i < j.$$

For example,

$$\begin{bmatrix} 2 & 0 & 0 \\ 3 & 4 & 0 \\ 4 & 8 & 1 \end{bmatrix}$$

is lower triangular.

Triangular matrices become especially useful when we solve linear systems. That topic appears later; for now, recognize the shape.

7.4 Transpose and Symmetry

The *transpose* of a matrix swaps rows and columns. If $A \in \mathbb{R}^{m \times n}$, then $A^T \in \mathbb{R}^{n \times m}$, and

$$(A^T)_{ij} = A_{ji}.$$

For example,

$$\begin{bmatrix} 0 & -4 \\ 7 & 0 \\ 3 & 1 \end{bmatrix}^T = \begin{bmatrix} 0 & 7 & 3 \\ -4 & 0 & 1 \end{bmatrix}.$$

The transpose also works with block matrices. If the block sizes are compatible, then

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix}^T = \begin{bmatrix} A^T & C^T \\ B^T & D^T \end{bmatrix}.$$

The blocks move across the diagonal, and each block is transposed.

A square matrix A is *symmetric* if

$$A^T = A.$$

Equivalently, $A_{ij} = A_{ji}$ for every pair of indices. Symmetric matrices appear often in least squares, covariance matrices, and quadratic forms.

7.5 Matrix Arithmetic

Two matrices of the same dimensions can be added or subtracted entry by entry. If $A, B \in \mathbb{R}^{m \times n}$, then

$$(A + B)_{ij} = A_{ij} + B_{ij}, \quad (A - B)_{ij} = A_{ij} - B_{ij}.$$

A scalar can multiply a matrix entry by entry:

$$(\alpha A)_{ij} = \alpha A_{ij}.$$

Matrix addition and scalar multiplication behave like the corresponding vector operations. The key shape rule is simple: addition and subtraction require matching dimensions. There is no standard mathematical operation that adds a 3×4 matrix to a 4×3 matrix, even though both contain 12 numbers.

If $A, B \in \mathbb{R}^{m \times n}$, then adding $A + B$ requires mn scalar additions. Multiplying A by a scalar also requires mn scalar multiplications. These operation counts are not important because of the exact constants; they are important because they help us think about how computation grows with matrix size.

7.6 Matrix-Vector Multiplication

Let

$$A \in \mathbb{R}^{m \times n}, \quad \mathbf{x} \in \mathbb{R}^n.$$

Then the product $A\mathbf{x}$ is defined, and the result is a vector in $\mathbb{R}^m$:

$$\mathbf{y} = A\mathbf{x}, \quad \mathbf{y} \in \mathbb{R}^m.$$

The inner dimensions must match: the number of columns of A must equal the number of entries in $\mathbf{x}$. The output length is the number of rows of A .

The source notes emphasize two complementary interpretations of matrix-vector multiplication.

7.6.1 Row Inner-Product View

The first interpretation computes each output entry as an inner product between $\mathbf{x}$ and one row of A . If $\mathbf{y} = A\mathbf{x}$, then

$$y_i = A_{i,:}\mathbf{x} = A_{i1}x_1 + A_{i2}x_2 + \cdots + A_{in}x_n.$$

Since $A_{i,:}$ is a row vector, this expression is the dot product of row i with $\mathbf{x}$.

For example, let

$$A = \begin{bmatrix} 0 & 2 & -1 \\ -2 & 1 & 1 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} 2 \\ 1 \\ -1 \end{bmatrix}.$$

Then

$$A\mathbf{x} = \begin{bmatrix} 0(2) + 2(1) + (-1)(-1) \\ -2(2) + 1(1) + 1(-1) \end{bmatrix} = \begin{bmatrix} 3 \\ -4 \end{bmatrix}.$$

The first entry comes from the first row of A , and the second entry comes from the second row of A .

7.6.2 Column Linear-Combination View

The second interpretation writes the output as a linear combination of the columns of A , using the entries of $\mathbf{x}$ as coefficients.

Let $A_{:,j}$ denote the j th column of A . Then

$$A\mathbf{x} = x_1A_{:,1} + x_2A_{:,2} + \cdots + x_nA_{:,n}.$$

Using the same example,

$$A = \begin{bmatrix} 0 & 2 & -1 \\ -2 & 1 & 1 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} 2 \\ 1 \\ -1 \end{bmatrix},$$

we can compute

$$A\mathbf{x} = 2 \begin{bmatrix} 0 \\ -2 \end{bmatrix} + 1 \begin{bmatrix} 2 \\ 1 \end{bmatrix} + (-1) \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ -4 \end{bmatrix}.$$

This interpretation is often the more important one conceptually. The matrix supplies a list of column vectors. The vector $\mathbf{x}$ supplies the coefficients. The output $A\mathbf{x}$ is the linear combination produced by those coefficients.

7.7 Computational Cost and Shape Checking

For $A \in \mathbb{R}^{m \times n}$ and $\mathbf{x} \in \mathbb{R}^n$, computing $A\mathbf{x}$ requires roughly $2mn$ floating-point operations. Each of the m output entries is a dot product of length n , requiring about n multiplications and $n - 1$ additions.

The approximate count is

$$mn + m(n - 1) = 2mn - m \approx 2mn.$$

This is not a rule to memorize for its own sake. It teaches two practical habits:

1. check the dimensions before multiplying; and

2. remember that matrix operations can become expensive when the number of rows and columns is large.

In software, many bugs are shape bugs. If $A \in \mathbb{R}^{m \times n}$, then $A\mathbf{x}$ is defined only when $\mathbf{x} \in \mathbb{R}^n$. The result will be in $\mathbb{R}^m$. Before interpreting the output, verify that the product you computed has the shape you intended.

7.8 A Matrix-Vector Dynamics Example

Matrix-vector multiplication is especially useful for state-transition models. Suppose a population is divided into four states:

- susceptible;
- infected;
- recovered and immune;
- deceased.

Let

$$\mathbf{x}_t = \begin{bmatrix} S_t \\ I_t \\ R_t \\ D_t \end{bmatrix}$$

be the vector of population proportions in these four states at time t . For example,

$$\mathbf{x}_t = \begin{bmatrix} 0.75 \\ 0.10 \\ 0.10 \\ 0.05 \end{bmatrix}$$

means that 75% of the population is susceptible, 10% infected, 10% recovered, and 5% deceased.

The source notes describe a simple transition model. Suppose:

- 95% of susceptible individuals remain susceptible and 5% become infected;
- of the infected individuals, 4% recover without immunity and return to susceptible, 85% remain infected, 10% recover with immunity, and 1% die;
- recovered immune individuals remain recovered;
- deceased individuals remain deceased.

Then the next state can be written as

$$\mathbf{x}_{t+1} = \begin{bmatrix} 0.95 & 0.04 & 0 & 0 \\ 0.05 & 0.85 & 0 & 0 \\ 0 & 0.10 & 1 & 0 \\ 0 & 0.01 & 0 & 1 \end{bmatrix} \mathbf{x}_t.$$

The matrix encodes how much of each current state flows into each future state.

This example illustrates why the column interpretation matters. Each column describes what happens to one current state. The state vector $\mathbf{x}_t$ provides the proportions in the current states. Multiplying the transition matrix by $\mathbf{x}_t$ combines the columns according to those proportions to produce the next state.

7.9 Data Analysis Bridge

The Data Analysis textbook uses the matrix ideas in this chapter whenever it writes a data table, design matrix, fitted-value equation, or linear model in compact form.

In regression, the design matrix X contains one row per observation and one column per model feature. The coefficient vector β supplies the weights. The product

$$X\beta$$

produces the vector of systematic fitted values. In the row view, each fitted value is the inner product of one observation's row with the coefficient vector. In the column view, the fitted vector is a linear combination of the columns of the design matrix.

The same two views appear throughout modeling:

- rows correspond to observations or equations;
- columns correspond to variables, features, constraints, or directions;
- transposes let row-column relationships be written cleanly;
- identity matrices appear in residual operators such as $I - H$;
- matrix-vector multiplication turns coefficient vectors into fitted-value vectors.

Later chapters use these ideas to discuss linear systems, QR factorization, inverses, rank, subspaces, least squares, normal equations, and projection. The mechanics begin here: a matrix is a structured collection of rows and columns, and multiplying by a vector creates a linear combination of those columns.

7.10 Chapter Summary

By the end of this chapter, you should be able to:

- identify the dimensions of a matrix and interpret entries A_{ij} ;
- distinguish square, tall, and wide matrices;
- extract rows, columns, blocks, and submatrices;
- recognize zero, identity, sparse, triangular, transpose, and symmetric matrices;
- check whether a matrix-vector product is dimensionally valid;

- compute $A\mathbf{x}$ using the row inner-product view;
- compute $A\mathbf{x}$ using the column linear-combination view; and
- explain why matrix-vector multiplication is central to data tables, state-transition models, and regression design matrices.

Chapter 8

Linear Systems, Polynomial Systems, and Vandermonde Matrices

The previous chapter introduced matrices and matrix-vector multiplication. We saw that a matrix can organize information, and that multiplying a matrix by a vector produces a linear combination of the matrix's columns. We now use that language to study one of the central problems in applied mathematics and operations research:

$$A\mathbf{x} = \mathbf{b}.$$

A linear system asks us to find the unknown vector $\mathbf{x}$ whose image under the matrix A is the known vector $\mathbf{b}$. This form appears whenever equations are written compactly, whenever polynomial coefficients are solved from data, and later when least squares and optimization problems are reduced to matrix equations.

This chapter has two goals. First, it explains how simple linear systems can be solved when the coefficient matrix has triangular structure. Second, it shows how polynomial interpolation leads naturally to a special matrix called a Vandermonde matrix. We keep the focus narrow: QR factorizations, matrix inverses, rank, pivoting, subspaces, and least squares all appear later. Here the goal is to understand what a linear system is and why polynomial fitting can be written as one.

8.1 Linear Systems in Matrix Form

A *linear system* is a collection of linear equations in unknowns. For example,

$$\begin{aligned} 2x_1 + 3x_2 &= 1, \\ 4x_2 &= 1 \end{aligned}$$

can be written in matrix form as

$$\begin{bmatrix} 2 & 3 \\ 0 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

The matrix contains the coefficients of the unknowns, the vector $\mathbf{x}$ contains the unknowns, and the vector $\mathbf{b}$ contains the right-hand side values.

More generally, if

$$A \in \mathbb{R}^{m \times n}, \quad \mathbf{x} \in \mathbb{R}^n, \quad \mathbf{b} \in \mathbb{R}^m,$$

then

$$A\mathbf{x} = \mathbf{b}$$

represents m scalar equations in n unknowns. Solving the system means finding a vector $\mathbf{x}$ that satisfies all of those equations at once.

The shape of A matters. A square system has the same number of equations and unknowns. A tall system has more equations than unknowns. A wide system has fewer equations than unknowns. The shape does not by itself tell us whether a solution exists or whether it is unique, but it tells us what kind of problem we are facing.

A useful interpretation comes from the column view of matrix-vector multiplication. If

$$A = [\mathbf{a}_1 \quad \mathbf{a}_2 \quad \cdots \quad \mathbf{a}_n],$$

then

$$A\mathbf{x} = x_1\mathbf{a}_1 + x_2\mathbf{a}_2 + \cdots + x_n\mathbf{a}_n.$$

Thus the equation $A\mathbf{x} = \mathbf{b}$ asks whether $\mathbf{b}$ can be built as a linear combination of the columns of A , and if so, which coefficients produce it.

8.2 Triangular Systems

Some linear systems are easier to solve because the coefficient matrix has zeros in useful places. An *upper triangular* matrix has zeros below the main diagonal. A *lower triangular* matrix has zeros above the main diagonal.

For example,

$$R = \begin{bmatrix} 2 & 3 \\ 0 & 4 \end{bmatrix}$$

is upper triangular. The system

$$R\mathbf{z} = \mathbf{b}, \quad \begin{bmatrix} 2 & 3 \\ 0 & 4 \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

is easy because the second equation has only one unknown:

$$4z_2 = 1, \quad z_2 = \frac{1}{4}.$$

Once z_2 is known, the first equation gives

$$2z_1 + 3z_2 = 1,$$

so

$$2z_1 = 1 - 3z_2 = 1 - \frac{3}{4} = \frac{1}{4}, \quad z_1 = \frac{1}{8}.$$

The order matters. We solved the last equation first, then moved upward.

8.3 Back Substitution

The procedure just used is called *back substitution*. It solves an upper triangular system by starting with the last equation and moving backward.

Let $R \in \mathbb{R}^{n \times n}$ be upper triangular with nonzero diagonal entries, and suppose we want to solve

$$R\mathbf{x} = \mathbf{b}.$$

Because R is upper triangular, equation i has the form

$$R_{ii}x_i + R_{i,i+1}x_{i+1} + \cdots + R_{in}x_n = b_i.$$

If we already know $x_{i+1}, \dots, x_n$, then we can solve for x_i :

$$x_i = \frac{b_i - \sum_{j=i+1}^n R_{ij}x_j}{R_{ii}}.$$

Back substitution applies this formula for

$$i = n, n-1, \dots, 1.$$

The nonzero diagonal condition matters. If some $R_{ii} = 0$, then the formula attempts to divide by zero, and this simple version of the algorithm breaks down.

Back substitution is computationally cheaper than solving a completely unstructured system from scratch. The exact count depends on implementation, but the source notes emphasize that the work grows on the order of n^2 operations for an $n \times n$ triangular system. This is much smaller than the work required to compute many full matrix factorizations.

8.4 Forward Substitution

Forward substitution is the corresponding method for lower triangular systems. Suppose

$$L\mathbf{x} = \mathbf{b}$$

where $L \in \mathbb{R}^{n \times n}$ is lower triangular with nonzero diagonal entries. The first equation contains only x_1 , so we solve for x_1 first. Then the second equation involves x_1 and x_2 , so we use the value of x_1 to solve for x_2 , and so on.

For a lower triangular system, equation i has the form

$$L_{i1}x_1 + L_{i2}x_2 + \cdots + L_{ii}x_i = b_i.$$

If $x_1, \dots, x_{i-1}$ are already known, then

$$x_i = \frac{b_i - \sum_{j=1}^{i-1} L_{ij}x_j}{L_{ii}}.$$

Forward substitution applies this formula for

$$i = 1, 2, \dots, n.$$

Together, forward and back substitution are the basic computational tools behind many larger methods. Later, when a factorization turns a hard system into triangular systems, these substitution procedures do the final solve.

8.5 Solving Systems by Reducing to Triangular Form

The source notes motivate triangular systems through factorizations. A factorization rewrites a matrix as a product of simpler matrices. Once that happens, a system can sometimes be solved in easier stages.

For example, suppose a matrix has been factored as

$$A = QR,$$

where the columns of Q are orthonormal and R is upper triangular. This is only a preview of why triangular solves matter; the construction and theory of QR are deferred to the next chapter. For now, assume the factorization has already been found.

If we want to solve

$$A\mathbf{x} = \mathbf{b},$$

then substituting $A = QR$ gives

$$QR\mathbf{x} = \mathbf{b}.$$

Multiplying both sides by Q^T gives

$$Q^T QR\mathbf{x} = Q^T \mathbf{b}.$$

When $Q^T Q = I$, this reduces to

$$R\mathbf{x} = Q^T \mathbf{b}.$$

Now the remaining system is upper triangular, so it can be solved by back substitution.

The important idea is not the QR algorithm itself. The important idea is the workflow:

$$\text{hard system} \longrightarrow \text{structured system} \longrightarrow \text{substitution solve}.$$

This pattern reappears throughout numerical linear algebra.

8.6 Polynomial Interpolation as a Linear System

The source notes next use a running example to show that polynomial fitting can be written as a linear system. Suppose we observe n data points

$$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n).$$

If the x_i values are distinct, then there is a unique polynomial of degree at most $n - 1$ that passes through all n points.

Write that polynomial as

$$p(x) = a_0 + a_1x + a_2x^2 + \dots + a_{n-1}x^{n-1}.$$

The unknowns are the coefficients

$$a_0, a_1, \dots, a_{n-1}.$$

To force the polynomial to pass through the data, we impose

$$p(x_i) = y_i, \quad i = 1, 2, \dots, n.$$

That gives the system

$$\begin{aligned} a_0 + a_1x_1 + a_2x_1^2 + \cdots + a_{n-1}x_1^{n-1} &= y_1, \\ a_0 + a_1x_2 + a_2x_2^2 + \cdots + a_{n-1}x_2^{n-1} &= y_2, \\ &\vdots \\ a_0 + a_1x_n + a_2x_n^2 + \cdots + a_{n-1}x_n^{n-1} &= y_n. \end{aligned}$$

This is a linear system in the unknown coefficients, even though the polynomial may be nonlinear as a function of x . The coefficients $a_0, \dots, a_{n-1}$ appear linearly.

This distinction is important: powers of the observed input values are allowed in the columns, but the unknown coefficients still enter as a linear combination.

8.7 Vandermonde Matrices

The polynomial interpolation system can be written compactly using a special matrix. Define

$$V = \begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^{n-1} \\ 1 & x_2 & x_2^2 & \cdots & x_2^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^{n-1} \end{bmatrix} \in \mathbb{R}^{n \times n}.$$

Let

$$\mathbf{a} = \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}.$$

Then the interpolation equations become

$$V\mathbf{a} = \mathbf{y}.$$

The matrix V is called a *Vandermonde matrix*. Each row is built from consecutive powers of one input value:

$$1, x_i, x_i^2, \dots, x_i^{n-1}.$$

Vandermonde matrices appear naturally in polynomial interpolation and in systems of equations involving powers of variables.

The source notes use a running-distance example: distance values become the inputs x_i , observed durations or paces become the outputs y_i , and the polynomial coefficients are found by solving $V\mathbf{a} = \mathbf{y}$. The example is not mainly about running. It is about recognizing that a curve-fitting problem can become a matrix equation.

A caution. Solving $V\mathbf{a} = \mathbf{y}$ gives the polynomial that passes through the data points when the inputs are distinct. That does not mean the polynomial is automatically a good predictive model. A high-degree polynomial can pass exactly through the observed data and still behave poorly between or beyond the observed points. This modeling caution becomes important later in data analysis when we distinguish interpolation, least squares, and prediction.

8.8 Exact Systems and Approximate Systems

Sometimes we do not want a polynomial that passes exactly through every point. Instead, we may choose a simpler polynomial, such as a quadratic:

$$p(x) = a_0 + a_1x + a_2x^2.$$

For n observations, the corresponding system has the form

$$\begin{bmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ \vdots & \vdots & \vdots \\ 1 & x_n & x_n^2 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} \approx \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}.$$

When $n > 3$, there are more equations than unknown coefficients. In general, we should not expect a single quadratic polynomial to pass through every observed point exactly.

This is where approximation enters. Instead of solving

$$A\mathbf{x} = \mathbf{b}$$

exactly, we may need to find $\mathbf{x}$ so that

$$A\mathbf{x} \approx \mathbf{b}.$$

That idea motivates least squares. The full theory is postponed to later chapters; the point here is narrower: data often give us more equations than unknowns, so exact equality may be too much to ask.

8.9 Data Analysis Bridge

The Data Analysis textbook uses the ideas in this chapter whenever transformed regression models are written in matrix form. If a model includes polynomial terms such as

$$1, \quad x, \quad x^2, \quad x^3,$$

then the design matrix contains columns built from powers of the observed predictor values. This is the same structural idea as a Vandermonde matrix.

For example, a polynomial regression model

$$y_i \approx a_0 + a_1x_i + a_2x_i^2$$

can be written as

$$\mathbf{y} \approx \begin{bmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ \vdots & \vdots & \vdots \\ 1 & x_n & x_n^2 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix}.$$

The design columns may be nonlinear functions of the original predictor values, but the model is still linear in the unknown coefficients.

This chapter also supports the computational habit of reducing problems to linear systems and solving them with structure. Later chapters develop QR, inverses, rank, subspaces, and least squares; this chapter provides the basic equation form those tools act on.

8.10 Chapter Summary

By the end of this chapter, you should be able to:

- write a system of linear equations in the form $A\mathbf{x} = \mathbf{b}$;
- interpret $A\mathbf{x} = \mathbf{b}$ as a question about linear combinations of the columns of A ;
- solve a small upper triangular system by back substitution;
- describe the corresponding forward substitution idea for lower triangular systems;
- explain why matrix factorizations can reduce hard systems to triangular systems;
- write polynomial interpolation conditions as a linear system;
- recognize a Vandermonde matrix and explain why it appears in polynomial fitting; and
- distinguish exact interpolation from approximate fitting.

Chapter 9

Orthogonal Bases, Gram-Schmidt, and QR

Chapter 6 introduced bases as linearly independent collections that can represent every vector in a space. Chapter 8 showed that structured linear systems are easier to solve when the matrix has a helpful shape, especially triangular shape. This chapter connects those two ideas.

The goal is to turn a linearly independent collection of vectors into a more convenient collection of vectors. The new vectors should point in mutually perpendicular directions and have length one. Such vectors are easier to use because their inner products behave like coordinate selectors. The main tool is the *Gram-Schmidt algorithm*. Applied to the columns of a matrix, the same idea produces a *QR factorization*

$$A = QR,$$

where the columns of Q are orthonormal and R is upper triangular.

This chapter stays focused on construction and interpretation. Later chapters handle inverses, rank, pseudo-inverses, column and null spaces, and least-squares theory. Here the key question is:

How can we replace a useful but non-orthogonal set of vectors by an orthonormal set that spans the same directions?

9.1 Orthogonal and Orthonormal Sets

Two nonzero vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ are *orthogonal* when

$$\mathbf{u}^T \mathbf{v} = 0.$$

Geometrically, orthogonal vectors meet at a right angle. Algebraically, orthogonality means that the dot product sees no component of one vector in the direction of the other.

A collection of vectors

$$\mathbf{q}_1, \mathbf{q}_2, \dots, \mathbf{q}_k$$

is an *orthogonal set* if

$$\mathbf{q}_i^T \mathbf{q}_j = 0 \quad \text{whenever } i \neq j.$$

The set is *orthonormal* if it is orthogonal and each vector has length one:

$$\|\mathbf{q}_i\| = 1, \quad i = 1, 2, \dots, k.$$

Equivalently,

$$\mathbf{q}_i^T \mathbf{q}_j = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases}$$

An *orthonormal basis* is a basis whose vectors are orthonormal. The standard basis vectors

$$\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_n$$

are the simplest example. They are mutually orthogonal, each has length one, and together they form a basis for $\mathbb{R}^n$.

The advantage of an orthonormal basis is computational clarity. If

$$\mathbf{x} = c_1 \mathbf{q}_1 + c_2 \mathbf{q}_2 + \dots + c_k \mathbf{q}_k$$

and the $\mathbf{q}_i$'s are orthonormal, then multiplying by $\mathbf{q}_j^T$ isolates the j th coefficient:

$$\mathbf{q}_j^T \mathbf{x} = c_j.$$

The inner product acts like a coordinate extractor because all cross terms vanish.

9.2 The Goal of Gram-Schmidt

Suppose we are given vectors

$$\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_k \in \mathbb{R}^n.$$

If these vectors are linearly independent, they form a basis for the set of directions they span. They may still be awkward to work with, because they need not be orthogonal and they need not have length one.

The Gram-Schmidt algorithm takes the input vectors $\mathbf{a}_1, \dots, \mathbf{a}_k$ and tries to produce vectors

$$\mathbf{q}_1, \mathbf{q}_2, \dots, \mathbf{q}_k$$

with three properties:

1. the $\mathbf{q}_i$'s are orthonormal;
2. each $\mathbf{q}_i$ is built from the vectors $\mathbf{a}_1, \dots, \mathbf{a}_i$;
3. the original directions are preserved: after step i , the vectors $\mathbf{q}_1, \dots, \mathbf{q}_i$ span the same set as $\mathbf{a}_1, \dots, \mathbf{a}_i$.

If the input vectors are linearly dependent, the algorithm detects this. At some step, the part of the new vector that is not already explained by the previous directions becomes the zero vector. That is the signal that no new direction has been added.

9.3 Removing Previously Used Directions

Assume that $\mathbf{q}_1, \dots, \mathbf{q}_{i-1}$ have already been constructed and are orthonormal. The next input vector is $\mathbf{a}_i$. To make a new orthogonal direction, remove from $\mathbf{a}_i$ all components lying in the directions already accounted for by $\mathbf{q}_1, \dots, \mathbf{q}_{i-1}$.

For an orthonormal vector $\mathbf{q}_j$, the amount of $\mathbf{a}_i$ in the $\mathbf{q}_j$ direction is

$$\mathbf{q}_j^T \mathbf{a}_i.$$

Therefore the component of $\mathbf{a}_i$ in the $\mathbf{q}_j$ direction is

$$(\mathbf{q}_j^T \mathbf{a}_i) \mathbf{q}_j.$$

Subtracting all previous components gives

$$\tilde{\mathbf{q}}_i = \mathbf{a}_i - \sum_{j=1}^{i-1} (\mathbf{q}_j^T \mathbf{a}_i) \mathbf{q}_j.$$

For $i = 1$, there are no previous directions, so

$$\tilde{\mathbf{q}}_1 = \mathbf{a}_1.$$

The vector $\tilde{\mathbf{q}}_i$ is the part of $\mathbf{a}_i$ that remains after subtracting all previously constructed orthonormal directions. If

$$\tilde{\mathbf{q}}_i = \mathbf{0},$$

then $\mathbf{a}_i$ was already a linear combination of the earlier vectors. The algorithm stops because the input set is linearly dependent.

If $\tilde{\mathbf{q}}_i \neq \mathbf{0}$, normalize it:

$$\mathbf{q}_i = \frac{\tilde{\mathbf{q}}_i}{\|\tilde{\mathbf{q}}_i\|}.$$

This gives a unit vector in the new direction.

9.4 The Gram-Schmidt Algorithm

The full algorithm can be written as follows.

1. Start with input vectors $\mathbf{a}_1, \dots, \mathbf{a}_k \in \mathbb{R}^n$.

2. For $i = 1, 2, \dots, k$, compute

$$\tilde{\mathbf{q}}_i = \mathbf{a}_i - \sum_{j=1}^{i-1} (\mathbf{q}_j^T \mathbf{a}_i) \mathbf{q}_j.$$

3. If $\tilde{\mathbf{q}}_i = \mathbf{0}$, stop. The input vectors are linearly dependent.

4. Otherwise set

$$\mathbf{q}_i = \frac{\tilde{\mathbf{q}}_i}{\|\tilde{\mathbf{q}}_i\|}.$$

The source notes emphasize the practical interpretation. Gram-Schmidt returns orthonormal vectors built from the original vectors. It also detects linear dependence when no new direction remains.

The computational work for classical Gram-Schmidt is on the order of

$$2nk^2$$

floating-point operations for k input vectors in $\mathbb{R}^n$. The exact count depends on implementation details, but the key message is that the cost grows with the ambient dimension n and roughly quadratically with the number of input vectors k .

9.5 Why the New Vector Is Orthogonal to the Old Ones

The subtraction step was designed to make $\tilde{\mathbf{q}}_i$ orthogonal to all previous $\mathbf{q}_j$'s. To see why, fix $j < i$. Then

$$\tilde{\mathbf{q}}_i = \mathbf{a}_i - \sum_{\ell=1}^{i-1} (\mathbf{q}_\ell^T \mathbf{a}_i) \mathbf{q}_\ell.$$

Take the dot product with $\mathbf{q}_j$:

$$\mathbf{q}_j^T \tilde{\mathbf{q}}_i = \mathbf{q}_j^T \mathbf{a}_i - \sum_{\ell=1}^{i-1} (\mathbf{q}_\ell^T \mathbf{a}_i) (\mathbf{q}_j^T \mathbf{q}_\ell).$$

Because the previous vectors are orthonormal,

$$\mathbf{q}_j^T \mathbf{q}_\ell = \begin{cases} 1, & \ell = j, \\ 0, & \ell \neq j. \end{cases}$$

All terms in the sum vanish except the one with $\ell = j$. Thus

$$\mathbf{q}_j^T \tilde{\mathbf{q}}_i = \mathbf{q}_j^T \mathbf{a}_i - (\mathbf{q}_j^T \mathbf{a}_i) = 0.$$

So $\tilde{\mathbf{q}}_i$ is orthogonal to every previous $\mathbf{q}_j$. Normalizing it changes its length, but not its direction, so $\mathbf{q}_i$ remains orthogonal to the previous vectors.

9.6 A Two-Vector Example

Consider the vectors

$$\mathbf{a}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \mathbf{a}_2 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

They are linearly independent, but not orthogonal, since

$$\mathbf{a}_1^T \mathbf{a}_2 = 1.$$

The first Gram-Schmidt vector is

$$\tilde{\mathbf{q}}_1 = \mathbf{a}_1, \quad \mathbf{q}_1 = \frac{\mathbf{a}_1}{\|\mathbf{a}_1\|} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

For the second vector, remove the component of $\mathbf{a}_2$ in the $\mathbf{q}_1$ direction:

$$\tilde{\mathbf{q}}_2 = \mathbf{a}_2 - (\mathbf{q}_1^T \mathbf{a}_2) \mathbf{q}_1.$$

Since

$$\mathbf{q}_1^T \mathbf{a}_2 = \frac{1}{\sqrt{2}},$$

we get

$$\tilde{\mathbf{q}}_2 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} - \frac{1}{\sqrt{2}} \left(\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right) = \begin{bmatrix} 1/2 \\ -1/2 \end{bmatrix}.$$

The norm of this vector is

$$\|\tilde{\mathbf{q}}_2\| = \sqrt{\left(\frac{1}{2}\right)^2 + \left(-\frac{1}{2}\right)^2} = \frac{1}{\sqrt{2}}.$$

Therefore

$$\mathbf{q}_2 = \frac{\tilde{\mathbf{q}}_2}{\|\tilde{\mathbf{q}}_2\|} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

Now

$$\mathbf{q}_1^T \mathbf{q}_2 = 0, \quad \|\mathbf{q}_1\| = \|\mathbf{q}_2\| = 1.$$

Thus Gram-Schmidt has replaced the original non-orthogonal pair by an orthonormal pair spanning the same plane.

9.7 Matrices with Orthonormal Columns

Place the orthonormal vectors into a matrix:

$$Q = [\mathbf{q}_1 \quad \mathbf{q}_2 \quad \cdots \quad \mathbf{q}_k] \in \mathbb{R}^{n \times k}.$$

The entries of $Q^T Q$ are dot products of the columns of Q :

$$(Q^T Q)_{ij} = \mathbf{q}_i^T \mathbf{q}_j.$$

Because the columns are orthonormal,

$$Q^T Q = I_k.$$

If Q is square and has orthonormal columns, then Q is called an *orthogonal matrix*. In that special square case,

$$Q^T Q = I \quad \text{and} \quad Q Q^T = I.$$

When $Q \in \mathbb{R}^{n \times k}$ is rectangular with $k < n$, the columns can still be orthonormal and still satisfy $Q^T Q = I_k$, but $Q Q^T$ is not the identity matrix on all of $\mathbb{R}^n$. This distinction will matter later when we discuss subspaces and projections.

9.8 Tall or Square Matrices with Independent Columns

Suppose

$$A \in \mathbb{R}^{n \times k}$$

has linearly independent columns. Then $k \leq n$. In words, a matrix with linearly independent columns must be tall or square.

The reason is the dimension bound from Chapter 6: a linearly independent set of vectors in $\mathbb{R}^n$ can have at most n elements. The columns of A are k vectors in $\mathbb{R}^n$. If $k > n$, then there are too many vectors for all of them to be linearly independent.

This does not yet use the formal language of rank. Rank comes later. For now, the important conclusion is simple:

$$\text{independent columns} \quad \Rightarrow \quad \text{not more columns than rows.}$$

9.9 From Gram-Schmidt to QR Factorization

Now let

$$A = [\mathbf{a}_1 \quad \mathbf{a}_2 \quad \cdots \quad \mathbf{a}_k] \in \mathbb{R}^{n \times k}$$

have linearly independent columns. Apply Gram-Schmidt to the columns of A . The result is an orthonormal collection

$$\mathbf{q}_1, \mathbf{q}_2, \dots, \mathbf{q}_k.$$

Let

$$Q = [\mathbf{q}_1 \quad \mathbf{q}_2 \quad \cdots \quad \mathbf{q}_k].$$

During the Gram-Schmidt construction, each original column $\mathbf{a}_i$ can be reconstructed from the first i orthonormal columns:

$$\mathbf{a}_i = R_{1i}\mathbf{q}_1 + R_{2i}\mathbf{q}_2 + \cdots + R_{ii}\mathbf{q}_i.$$

The coefficients are

$$R_{ji} = \mathbf{q}_j^T \mathbf{a}_i \quad \text{for } j < i, \quad R_{ii} = \|\tilde{\mathbf{q}}_i\|.$$

There are no components of $\mathbf{a}_i$ in $\mathbf{q}_{i+1}, \dots, \mathbf{q}_k$, so

$$R_{ji} = 0 \quad \text{for } j > i.$$

Thus the matrix of coefficients

$$R = \begin{bmatrix} R_{11} & R_{12} & R_{13} & \cdots & R_{1k} \\ 0 & R_{22} & R_{23} & \cdots & R_{2k} \\ 0 & 0 & R_{33} & \cdots & R_{3k} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & R_{kk} \end{bmatrix}$$

is upper triangular. Stacking the column equations gives

$$A = QR.$$

This is the *QR factorization*. In the common “thin QR” case,

$$Q \in \mathbb{R}^{n \times k}, \quad R \in \mathbb{R}^{k \times k}, \quad Q^T Q = I_k.$$

The factorization separates two jobs. The matrix Q stores orthonormal directions. The matrix R stores the coefficients needed to rebuild the original columns of A from those directions.

9.10 Using QR to Solve a Linear System

Chapter 8 showed that upper triangular systems can be solved by back substitution. QR connects a general matrix equation to that easier triangular form.

Suppose $A \in \mathbb{R}^{n \times k}$ has independent columns and the exact system

$$A\mathbf{x} = \mathbf{b}$$

is consistent, so $\mathbf{b}$ lies in the span of the columns of Q . Using the QR factorization

$$A = QR.$$

we have

$$QR\mathbf{x} = \mathbf{b}.$$

Multiplying both sides by Q^T gives

$$Q^T QR\mathbf{x} = Q^T \mathbf{b}.$$

Since $Q^T Q = I_k$, this becomes

$$R\mathbf{x} = Q^T \mathbf{b}.$$

Now the coefficient matrix is upper triangular, so $\mathbf{x}$ can be found by back substitution.

This is the same workflow previewed in Chapter 8:

$$A\mathbf{x} = \mathbf{b} \quad \longrightarrow \quad QR\mathbf{x} = \mathbf{b} \quad \longrightarrow \quad R\mathbf{x} = Q^T \mathbf{b}.$$

The full least-squares version of this idea, where the system may not have an exact solution, is deferred to Chapter 12. For now, QR should be understood as a way to convert a difficult-looking system into a triangular solve after constructing orthonormal columns.

9.11 Orthogonal-Polynomial Intuition

Chapter 8 introduced polynomial interpolation through columns such as

$$1, \quad x, \quad x^2, \quad \dots, \quad x^d.$$

When these columns are evaluated at observed input values, they form columns of a matrix. These columns are usually not orthogonal. In fact, polynomial columns can be strongly related to each other, especially when powers of x grow large.

Gram-Schmidt gives a way to build new columns from the old ones so that the new columns are orthonormal while preserving the same span. This is the basic intuition behind *orthogonal polynomial* features. Instead of using raw polynomial columns directly, one can replace them with orthogonalized columns built from the same polynomial information.

The point here is not to develop a full theory of orthogonal polynomials. The point is the construction principle:

$$\text{raw polynomial columns} \quad \xrightarrow{\text{Gram-Schmidt}} \quad \text{orthonormal polynomial-like columns}.$$

Later, in regression, this matters because changing the columns of a design matrix changes the coordinates used for computation without necessarily changing the set of fitted patterns that the columns can represent.

9.12 Data Analysis Bridge

The Data Analysis textbook uses this chapter mainly in three ways.

First, orthonormal columns explain why dot products can behave like coordinate selectors. This supports the geometric interpretation of fitted values, residuals, and transformed feature columns.

Second, Gram-Schmidt explains how a non-orthogonal set of useful columns can be converted into an orthonormal set without throwing away the directions those columns represent. This is the mathematical background for orthogonalized features and orthogonal-polynomial intuition.

Third, QR factorization is one reason numerical linear algebra often uses orthogonal transformations and triangular solves instead of inverse formulas. Later chapters revisit this idea when discussing inverses, rank, subspaces, and least squares.

9.13 Chapter Summary

The main ideas from this chapter are:

- Orthogonal vectors have dot product zero.
- Orthonormal vectors are orthogonal and have length one.
- If $Q \in \mathbb{R}^{n \times k}$ has orthonormal columns, then $Q^T Q = I_k$.
- Gram-Schmidt converts linearly independent input vectors into orthonormal vectors spanning the same directions.
- If a new vector contributes no new direction, Gram-Schmidt detects linear dependence. The signal is $\tilde{\mathbf{q}}_i = \mathbf{0}$.
- Applying Gram-Schmidt to the columns of a matrix gives the QR factorization $A = QR$.
- In QR, Q contains orthonormal columns and R is upper triangular.
- QR can reduce a system $A\mathbf{x} = \mathbf{b}$ to the triangular system $R\mathbf{x} = Q^T \mathbf{b}$.
- Orthogonal polynomial features are motivated by applying orthogonalization ideas to raw polynomial columns.

Chapter 10

Matrix Inverses, Rank, and Pseudoinverses

Chapter 8 introduced systems of equations

$$A\mathbf{x} = \mathbf{b},$$

and Chapter 9 showed how QR factorization can turn some matrix problems into triangular solves. This chapter asks when a matrix can be “undone.” For square matrices, the answer is the usual inverse. For rectangular matrices, the answer splits into two one-sided ideas: left inverses and right inverses.

The shape and rank of a matrix determine what kind of inverse-like behavior is possible. A tall matrix may have independent columns and a left inverse. A wide matrix may have independent rows and a right inverse. A square matrix that has both is invertible in the usual sense. These ideas are central later because regression design matrices are usually tall, and unique regression coefficients require full column rank.

This chapter focuses on inverse-like behavior, rank, and full-rank pseudoinverse formulas. Column space, row space, and null space are developed in the next chapter; least squares and normal equations appear later.

10.1 Left Inverses

Let $A \in \mathbb{R}^{m \times n}$. A matrix $C \in \mathbb{R}^{n \times m}$ is a *left inverse* of A if

$$CA = I_n.$$

It is called a left inverse because the undoing matrix appears on the left of A .

If $A\mathbf{x} = \mathbf{b}$ has a solution and C is a left inverse, then

$$A\mathbf{x} = \mathbf{b} \implies CA\mathbf{x} = C\mathbf{b} \implies \mathbf{x} = C\mathbf{b}.$$

Thus a left inverse gives a candidate solution

$$\hat{\mathbf{x}} = C\mathbf{b}.$$

The candidate must still be checked. If $A\hat{\mathbf{x}} = \mathbf{b}$, then $\hat{\mathbf{x}}$ is the solution. If $A\hat{\mathbf{x}} \neq \mathbf{b}$, then the system has no solution.

The key structural fact is

$$A \text{ has a left inverse} \iff \text{the columns of } A \text{ are linearly independent.}$$

To see the important direction, suppose $CA = I_n$ and $A\mathbf{x} = \mathbf{0}$. Then

$$CA\mathbf{x} = C\mathbf{0}, \quad \mathbf{x} = \mathbf{0}.$$

So the only solution to $A\mathbf{x} = \mathbf{0}$ is the trivial solution, which means the columns of A are linearly independent.

A left-invertible matrix must be square or tall. Since A has n independent columns in $\mathbb{R}^m$, we must have

$$m \geq n.$$

Left invertibility also gives uniqueness: if $A\mathbf{x} = \mathbf{b}$ has a solution, then it has only one. If $\mathbf{x}$ and $\hat{\mathbf{x}}$ both solve the system, then

$$A(\mathbf{x} - \hat{\mathbf{x}}) = \mathbf{0},$$

and column independence forces $\mathbf{x} - \hat{\mathbf{x}} = \mathbf{0}$.

10.2 Right Inverses

A matrix $C \in \mathbb{R}^{n \times m}$ is a *right inverse* of $A \in \mathbb{R}^{m \times n}$ if

$$AC = I_m.$$

Right inverses are tied to rows rather than columns. Since

$$AC = I_m \implies C^T A^T = I_m,$$

the transpose C^T is a left inverse of A^T . Therefore A^T has linearly independent columns, which means A has linearly independent rows. The converse is also true, so

$$A \text{ has a right inverse} \iff \text{the rows of } A \text{ are linearly independent.}$$

A right-invertible matrix must be square or wide:

$$m \leq n.$$

Right invertibility guarantees existence, not uniqueness. If $AC = I_m$, then for any $\mathbf{b} \in \mathbb{R}^m$,

$$\hat{\mathbf{x}} = C\mathbf{b}$$

satisfies

$$A\hat{\mathbf{x}} = AC\mathbf{b} = \mathbf{b}.$$

So every right-hand side is reachable. But if the system is wide, there may be many different vectors $\mathbf{x}$ that produce the same $\mathbf{b}$.

10.3 Square Matrix Inverses

If a matrix has both a left inverse and a right inverse, then those inverse-like matrices must agree. Suppose

$$AX = I, \quad YA = I,$$

where X is a right inverse and Y is a left inverse. Then

$$X = IX = (YA)X = Y(AX) = YI = Y.$$

The common matrix is denoted A^{-1} . A square matrix with such an inverse is called *invertible* or *nonsingular*; a square matrix without an inverse is called *singular*.

For a square matrix $A \in \mathbb{R}^{n \times n}$, the following statements are equivalent:

1. A is invertible;
2. the columns of A are linearly independent;
3. the rows of A are linearly independent;
4. A has a left inverse;
5. A has a right inverse.

When A is invertible, every system $A\mathbf{x} = \mathbf{b}$ has exactly one solution:

$$\mathbf{x} = A^{-1}\mathbf{b}.$$

Existence comes from right invertibility; uniqueness comes from left invertibility.

10.4 Computing a Square Inverse with QR

Chapter 9 gave the factorization

$$A = QR,$$

where Q has orthonormal columns and R is upper triangular. If A is square and invertible, then the inverse can be found by solving

$$AX = I.$$

Substituting $A = QR$ gives

$$QRX = I.$$

Multiplying by Q^T gives

$$RX = Q^T.$$

This is a collection of triangular systems, one for each column of X . Thus QR reduces inverse computation to back substitution. The practical lesson is that factorizations often give better computational tools than explicitly forming inverse matrices.

10.5 Rank and Full Rank

Rank measures how many independent directions a matrix contains. For $A \in \mathbb{R}^{m \times n}$, the *column rank* is the number of linearly independent columns, and the *row rank* is the number of linearly independent rows.

A standard theorem says that row rank and column rank are equal. This common value is called the *rank* of A , written

$$\text{rank}(A).$$

Because an $m \times n$ matrix has only m rows and n columns,

$$\text{rank}(A) \leq \min(m, n).$$

A matrix has *full rank* if

$$\text{rank}(A) = \min(m, n).$$

There are two shape-dependent cases.

If $m \geq n$, full rank means

$$\text{rank}(A) = n.$$

Then all columns are independent, and A has *full column rank*. This is the condition for a left inverse.

If $m \leq n$, full rank means

$$\text{rank}(A) = m.$$

Then all rows are independent, and A has *full row rank*. This is the condition for a right inverse.

For a square matrix, full row rank and full column rank are the same condition, and both are equivalent to ordinary invertibility.

10.6 Why $A^T A$ Appears

For tall full-column-rank matrices, the matrix $A^T A$ is square and invertible. This fact explains the familiar expression

$$(A^T A)^{-1} A^T.$$

Suppose A has linearly independent columns. To show $A^T A$ is invertible, it is enough to show that

$$A^T A \mathbf{x} = \mathbf{0}$$

has only the trivial solution. Multiply on the left by $\mathbf{x}^T$:

$$\mathbf{x}^T A^T A \mathbf{x} = 0.$$

But

$$\mathbf{x}^T A^T A \mathbf{x} = (A \mathbf{x})^T (A \mathbf{x}) = \|A \mathbf{x}\|^2.$$

Therefore $A \mathbf{x} = \mathbf{0}$. Since the columns of A are independent, $\mathbf{x} = \mathbf{0}$. Thus $A^T A$ is invertible.

Conversely, if the columns of A are dependent, then some nonzero $\mathbf{x}$ satisfies $A \mathbf{x} = \mathbf{0}$, and therefore $A^T A \mathbf{x} = \mathbf{0}$. So $A^T A$ is not invertible.

Hence

$$A^T A \text{ is invertible} \iff A \text{ has linearly independent columns.}$$

The row version is analogous:

$$A A^T \text{ is invertible} \iff A \text{ has linearly independent rows.}$$

10.7 Full-Rank Pseudoinverse Formulas

The source notes use the word *pseudoinverse* for the following full-rank one-sided inverse formulas. We keep that convention here. More general pseudoinverse theory exists, but these full-rank cases are the formulas needed for this course.

If $A \in \mathbb{R}^{m \times n}$ has full column rank, then

$$A_L^+ = (A^T A)^{-1} A^T$$

is a left inverse, because

$$A_L^+ A = (A^T A)^{-1} A^T A = I_n.$$

If $A \in \mathbb{R}^{m \times n}$ has full row rank, then

$$A_R^+ = A^T (A A^T)^{-1}$$

is a right inverse, because

$$A A_R^+ = A A^T (A A^T)^{-1} = I_m.$$

These formulas match the solution behavior above: full column rank gives uniqueness when a solution exists, and full row rank gives existence for every right-hand side.

10.8 Computing Full-Rank Pseudoinverses with QR

QR also gives a computational route to the full-rank pseudoinverse formulas.

If $A \in \mathbb{R}^{m \times n}$ has full column rank and thin QR factorization $A = QR$, then

$$A_L^+ = (A^T A)^{-1} A^T = R^{-1} Q^T.$$

This follows from

$$A^T A = (QR)^T (QR) = R^T Q^T Q R = R^T R.$$

The source notes express this computationally as solving

$$R X = Q^T.$$

The solution X is the left pseudoinverse.

For the full-row-rank case, apply QR to the transpose:

$$A^T = QR.$$

Then

$$A_R^+ = A^T (A A^T)^{-1} = Q R^{-T}.$$

Here R^{-T} means $(R^T)^{-1}$. Equivalently, solve a triangular system involving R , then transpose as needed. The key computational idea is the same: use QR and triangular solves rather than explicitly inverting large matrices when possible.

10.9 Rank Determination with QR

QR can also help determine rank. In exact arithmetic:

- to determine column rank, perform QR on A and inspect the diagonal of R ;
- to determine row rank, perform QR on A^T and inspect the diagonal of R .

A zero diagonal entry indicates that a new independent direction has failed to appear. In numerical work, the practical question is whether a diagonal entry is small relative to the scale of the problem, not whether it is symbolically zero.

10.10 Data Analysis Bridge

In regression, the design matrix usually has more rows than columns:

$$X \in \mathbb{R}^{n \times (p+1)}.$$

The columns represent the intercept and predictor directions. For ordinary least squares coefficients to be uniquely identified, these columns must be linearly independent. In this chapter's language, X must have full column rank.

When X has full column rank, $X^T X$ is invertible, which is why the familiar coefficient formula

$$\hat{\beta} = (X^T X)^{-1} X^T \mathbf{y}$$

requires a rank condition. If a predictor column is an exact linear combination of other columns, then $X^T X$ is singular and the coefficient vector is not uniquely determined.

This is the mathematical basis for common regression warnings: do not include duplicate predictors, do not include all dummy variables plus an intercept, and be careful when transformed columns create exact redundancy. The next chapter develops the subspace language that explains these non-identifiable directions.

10.11 Chapter Summary

- A left inverse satisfies $CA = I_n$. It exists exactly when the columns of A are linearly independent.
- Left invertibility gives uniqueness when a solution exists, but not necessarily existence for every $\mathbf{b}$.
- A right inverse satisfies $AC = I_m$. It exists exactly when the rows of A are linearly independent.
- Right invertibility gives existence for every $\mathbf{b}$, but not necessarily uniqueness.
- A square matrix is invertible exactly when its rows and columns are independent.
- Rank is the common row/column independence count. Full rank means $\text{rank}(A) = \min(m, n)$.

- Full column rank is the tall-matrix condition for left invertibility. Full row rank is the wide-matrix condition for right invertibility.
- $A^T A$ is invertible exactly when A has independent columns. Similarly, AA^T is invertible exactly when A has independent rows.
- The full-column-rank pseudoinverse formula is $(A^T A)^{-1} A^T$. The full-row-rank formula is $A^T (AA^T)^{-1}$.
- QR factorization turns inverse and pseudoinverse computation into triangular solves.

Chapter 11

Subspaces: Column Space, Row Space, and Null Space

Chapter 10 introduced rank, full rank, and inverse-like behavior. This chapter gives the geometric language behind those facts. Instead of asking only whether a matrix has independent rows or columns, we ask where the outputs of the matrix can live, which coefficient directions are invisible to the matrix, and how these spaces fit together.

For a matrix $A \in \mathbb{R}^{m \times n}$, there are three spaces that matter immediately:

$$\text{Col}(A), \quad \text{Row}(A), \quad \text{Null}(A).$$

The column space describes what right-hand sides can be reached by multiplying by A . The row space describes the independent coefficient directions that the rows can detect. The null space describes the directions that A sends to zero. These ideas are especially important when a linear system is underdetermined or when a regression design matrix fails to identify a unique coefficient vector.

11.1 Subspaces

A *subspace* is a subset of a vector space that is itself a vector space. In this course, the most important vector spaces are $\mathbb{R}^n$ and $\mathbb{R}^m$. A subset $S \subseteq \mathbb{R}^n$ is a subspace if it is closed under linear combinations: whenever $\mathbf{x}, \mathbf{y} \in S$ and $\alpha, \beta \in \mathbb{R}$,

$$\alpha\mathbf{x} + \beta\mathbf{y} \in S.$$

This one closure rule includes both addition and scalar multiplication. For example, if S is a subspace, then taking $\alpha = 1$ and $\beta = 1$ shows that $\mathbf{x} + \mathbf{y} \in S$, while taking $\beta = 0$ shows that $\alpha\mathbf{x} \in S$. It also implies that the zero vector belongs to every subspace, since

$$0\mathbf{x} = \mathbf{0}.$$

A span is the basic source of subspaces. If $\mathbf{v}_1, \dots, \mathbf{v}_k \in \mathbb{R}^n$, then

$$\text{span}\{\mathbf{v}_1, \dots, \mathbf{v}_k\} = \{\beta_1\mathbf{v}_1 + \dots + \beta_k\mathbf{v}_k : \beta_1, \dots, \beta_k \in \mathbb{R}\}$$

is a subspace of $\mathbb{R}^n$. It contains every vector reachable by taking linear combinations of the given vectors.

11.2 Column Space

Let

$$A = [\mathbf{a}_1 \quad \mathbf{a}_2 \quad \cdots \quad \mathbf{a}_n] \in \mathbb{R}^{m \times n},$$

where $\mathbf{a}_1, \dots, \mathbf{a}_n \in \mathbb{R}^m$ are the columns of A . The *column space* of A is the span of its columns:

$$\text{Col}(A) = \text{span}\{\mathbf{a}_1, \dots, \mathbf{a}_n\}.$$

Equivalently,

$$\text{Col}(A) = \{A\mathbf{x} : \mathbf{x} \in \mathbb{R}^n\} \subseteq \mathbb{R}^m.$$

The equivalence comes from the column-combination interpretation of matrix-vector multiplication:

$$A\mathbf{x} = x_1\mathbf{a}_1 + x_2\mathbf{a}_2 + \cdots + x_n\mathbf{a}_n.$$

Thus the column space answers the question:

Which vectors in $\mathbb{R}^m$ can be produced as $A\mathbf{x}$?

If a linear system

$$A\mathbf{x} = \mathbf{b}$$

has a solution, then $\mathbf{b} \in \text{Col}(A)$. If $\mathbf{b} \notin \text{Col}(A)$, then no exact solution exists. In that case, later least-squares methods look for the vector in $\text{Col}(A)$ that is closest to $\mathbf{b}$.

The dimension of the column space is the number of linearly independent columns of A . This is the *column rank*. Since Chapter 10 used $\text{rank}(A)$ for the common row/column rank, we can write

$$\dim \text{Col}(A) = \text{rank}(A).$$

11.3 Row Space

The *row space* of A is the span of its rows. Since rows of A are columns of A^T , it is often cleanest to define

$$\text{Row}(A) = \text{Col}(A^T).$$

If $A \in \mathbb{R}^{m \times n}$, then each row has n entries, so

$$\text{Row}(A) \subseteq \mathbb{R}^n.$$

The source notes also express the row space as

$$\text{Row}(A) = \{A^T \mathbf{y} : \mathbf{y} \in \mathbb{R}^m\}.$$

The row space answers a different question than the column space. The column space describes reachable outputs. The row space describes the directions in the input or coefficient space that the matrix can actually detect.

The dimension of the row space is the number of linearly independent rows of A . This is the *row rank*. Since row rank and column rank are equal,

$$\dim \text{Row}(A) = \text{rank}(A).$$

11.4 Null Space

The *null space* of A is the set of all input vectors that are mapped to zero:

$$\text{Null}(A) = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} = \mathbf{0}\}.$$

This is a subspace of $\mathbb{R}^n$. To see this, suppose $\mathbf{x}, \mathbf{y} \in \text{Null}(A)$, so

$$A\mathbf{x} = \mathbf{0}, \quad A\mathbf{y} = \mathbf{0}.$$

For any $\alpha, \beta \in \mathbb{R}$,

$$A(\alpha\mathbf{x} + \beta\mathbf{y}) = \alpha A\mathbf{x} + \beta A\mathbf{y} = \alpha\mathbf{0} + \beta\mathbf{0} = \mathbf{0}.$$

Therefore $\alpha\mathbf{x} + \beta\mathbf{y} \in \text{Null}(A)$, so the null space is closed under linear combinations.

The null space answers the question:

Which nonzero input directions, if any, does the matrix fail to see?

This gives an immediate test for column independence:

$$A \text{ has linearly independent columns} \iff \text{Null}(A) = \{\mathbf{0}\}.$$

Indeed, the columns of A are linearly independent exactly when the equation

$$A\mathbf{x} = \mathbf{0}$$

has only the trivial solution $\mathbf{x} = \mathbf{0}$. If there is a nonzero vector in the null space, then the columns are linearly dependent.

Null spaces are important because they describe non-uniqueness. If $A\mathbf{x} = \mathbf{b}$ has one solution $\mathbf{x}_0$, and $\mathbf{z} \in \text{Null}(A)$, then

$$A(\mathbf{x}_0 + \mathbf{z}) = A\mathbf{x}_0 + A\mathbf{z} = \mathbf{b} + \mathbf{0} = \mathbf{b}.$$

So every null-space direction gives another solution. Thus the null space is the space of invisible changes to $\mathbf{x}$.

11.5 Left Null Space

The *left null space* of A is the null space of A^T :

$$\text{Null}(A^T) = \{\mathbf{y} \in \mathbb{R}^m : A^T\mathbf{y} = \mathbf{0}\}.$$

It is called the left null space because it is equivalent to the set of row-vector directions $\mathbf{y}^T$ satisfying

$$\mathbf{y}^T A = \mathbf{0}^T.$$

If the ordinary null space describes input directions that A sends to zero, then the left null space describes output-space directions that are orthogonal to every column of A .

This space appears naturally in least squares. When a vector $\mathbf{b} \in \mathbb{R}^m$ cannot be represented exactly as $A\mathbf{x}$, the least-squares residual $\mathbf{b} - A\hat{\mathbf{x}}$ belongs to the part of output space orthogonal to $\text{Col}(A)$. That idea becomes the geometric heart of least squares in Chapter 12.

11.6 Orthogonality Among the Subspaces

The row space and null space live in the same ambient space, $\mathbb{R}^n$. They are orthogonal to each other. To see why, take

$$\mathbf{r} \in \text{Row}(A), \quad \mathbf{z} \in \text{Null}(A).$$

Since $\mathbf{r} \in \text{Row}(A)$, there is some $\mathbf{y} \in \mathbb{R}^m$ such that $\mathbf{r} = A^T \mathbf{y}$. Then

$$\mathbf{r}^T \mathbf{z} = (A^T \mathbf{y})^T \mathbf{z} = \mathbf{y}^T A \mathbf{z} = \mathbf{y}^T \mathbf{0} = 0.$$

So every row-space direction is orthogonal to every null-space direction.

Similarly,

$$\text{Col}(A) \text{ is orthogonal to } \text{Null}(A^T).$$

If $\mathbf{c} = A\mathbf{x} \in \text{Col}(A)$ and $\mathbf{w} \in \text{Null}(A^T)$, then

$$\mathbf{w}^T \mathbf{c} = \mathbf{w}^T A\mathbf{x} = (A^T \mathbf{w})^T \mathbf{x} = \mathbf{0}^T \mathbf{x} = 0.$$

These orthogonality relationships organize the four fundamental subspaces.

11.7 The Four Fundamental Subspaces

Let $A \in \mathbb{R}^{m \times n}$ and let

$$r = \text{rank}(A).$$

The four fundamental subspaces are:

Subspace	Ambient space	Dimension
$\text{Col}(A)$	$\mathbb{R}^m$	r
$\text{Null}(A^T)$	$\mathbb{R}^m$	$m - r$
$\text{Row}(A)$	$\mathbb{R}^n$	r
$\text{Null}(A)$	$\mathbb{R}^n$	$n - r$

The first pair lives in output space $\mathbb{R}^m$: column-space directions and left-null-space directions. The second pair lives in input space $\mathbb{R}^n$: row-space directions and null-space directions.

The dimension statements are often summarized as the rank-nullity relationships:

$$\dim \text{Col}(A) + \dim \text{Null}(A^T) = m,$$

$$\dim \text{Row}(A) + \dim \text{Null}(A) = n.$$

This table is useful because it keeps two different spaces separate. Column space and left null space live where outputs live. Row space and null space live where inputs or coefficient vectors live.

11.8 Example: Non-Uniqueness in a Targeting Problem

The source notes use a targeting problem to illustrate how linear systems and hidden directions arise. Suppose a projectile state contains position, velocity, and acceleration. A simplified update can be written in block form as

$$\begin{bmatrix} \mathbf{x}_{t+1} \\ \mathbf{v}_{t+1} \\ \mathbf{a}_{t+1} \end{bmatrix} = \begin{bmatrix} I & \Delta t I & 0 \\ 0 & I & \Delta t I \\ 0 & 0 & I \end{bmatrix} \begin{bmatrix} \mathbf{x}_t \\ \mathbf{v}_t \\ \mathbf{a}_t \end{bmatrix}.$$

After many time steps, the final position can be written as a linear function of the initial state. If the only requirement is to hit a target position, then there may be fewer constraints than unknown initial-state quantities. That leads to an underdetermined linear system.

Subspace language explains what happens. One solution may hit the target, but adding any vector in the null space of the targeting matrix gives another solution that produces the same constrained output. In practical terms, the target may be fixed, while some parts of the initial velocity or other state variables remain free.

The details of the computation belong to a computational notebook. The mathematical lesson is the important part here:

When a linear system is underdetermined, the null space describes the directions that can change without changing the constrained output.

11.9 Data Analysis Bridge

Regression uses these same subspaces with a design matrix. If

$$X \in \mathbb{R}^{n \times (p+1)},$$

then X maps coefficient vectors in $\mathbb{R}^{p+1}$ to fitted-value vectors in observation space $\mathbb{R}^n$:

$$\hat{\mathbf{y}} = X\hat{\boldsymbol{\beta}}.$$

Thus fitted values live in

$$\text{Col}(X) \subseteq \mathbb{R}^n.$$

For least-squares fits, residual vectors are orthogonal to the fitted-value directions, so they live in the left null space:

$$\mathbf{e} \in \text{Null}(X^T).$$

The coefficient-space subspaces are just as important. The row space of X contains coefficient directions that the data can identify. The null space of X contains coefficient directions that do not change fitted values:

$$X(\boldsymbol{\beta} + \mathbf{z}) = X\boldsymbol{\beta} \quad \text{for every } \mathbf{z} \in \text{Null}(X).$$

So a nonzero null space means non-identifiability: multiple coefficient vectors give the same fitted responses. This is the linear algebra behind exact multicollinearity and duplicate predictors.

11.10 Chapter Summary

- A subspace is a subset closed under linear combinations.
- The column space $\text{Col}(A)$ is the span of the columns of A , equivalently $\{A\mathbf{x} : \mathbf{x} \in \mathbb{R}^n\}$.
- A system $A\mathbf{x} = \mathbf{b}$ is solvable only when $\mathbf{b} \in \text{Col}(A)$.
- The row space $\text{Row}(A)$ is the span of the rows of A , equivalently $\text{Col}(A^T)$.
- The null space $\text{Null}(A)$ is the set of vectors satisfying $A\mathbf{x} = \mathbf{0}$.
- The null space is a subspace, and $\text{Null}(A) = \{\mathbf{0}\}$ exactly when the columns of A are linearly independent.
- The left null space is $\text{Null}(A^T)$.
- The row space is orthogonal to the null space, and the column space is orthogonal to the left null space.
- If $r = \text{rank}(A)$, then the four fundamental dimensions are r , $m - r$, r , and $n - r$ for $\text{Col}(A)$, $\text{Null}(A^T)$, $\text{Row}(A)$, and $\text{Null}(A)$, respectively.
- In regression, fitted values live in $\text{Col}(X)$, residuals live in $\text{Null}(X^T)$, and non-identifiable coefficient directions live in $\text{Null}(X)$.

Chapter 12

Multivariable Functions, Gradients, and Least Squares

The previous chapters developed the linear algebra needed to describe systems of equations, inverse-like behavior, rank, and subspaces. We now return to calculus. The goal is not to develop a full optimization course. The goal is more focused: we need the derivative language that explains why least squares leads to the normal equations and why the residual vector is orthogonal to the columns of the matrix.

The main objects in this chapter are functions whose inputs are vectors. A function may take a vector and output a scalar,

$$f : \mathbb{R}^n \rightarrow \mathbb{R},$$

or it may take a vector and output another vector,

$$f : \mathbb{R}^n \rightarrow \mathbb{R}^m.$$

For scalar-valued functions, gradients point in directions of steepest increase. For vector-valued functions, derivatives are matrices. Those matrices are called Jacobians. Both ideas are needed for least squares and for the chain-rule calculations that appear in modern modeling methods.

12.1 Multivariable Functions

A multivariable function is a function whose input has more than one coordinate. For example,

$$f : \mathbb{R}^n \rightarrow \mathbb{R}$$

takes an input vector

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

and returns a scalar. A quadratic function such as

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T A \mathbf{x} + \mathbf{b}^T \mathbf{x}$$

is one example. These functions appear when we write optimization problems with vector decision variables.

A vector-valued function has the form

$$f : \mathbb{R}^n \rightarrow \mathbb{R}^m.$$

For such a function, each input vector produces an output vector with m entries. We can write

$$f(\mathbf{x}) = \begin{bmatrix} f_1(\mathbf{x}) \\ f_2(\mathbf{x}) \\ \vdots \\ f_m(\mathbf{x}) \end{bmatrix}.$$

A linear transformation $f(\mathbf{x}) = A\mathbf{x}$, where $A \in \mathbb{R}^{m \times n}$, is the most important first example.

12.2 Partial Derivatives

For a scalar-valued function

$$f : \mathbb{R}^n \rightarrow \mathbb{R},$$

the partial derivative with respect to x_i measures the rate of change of f as only the i th coordinate changes. Holding all other coordinates fixed,

$$\frac{\partial f}{\partial x_i}(\mathbf{x}) = \lim_{h \rightarrow 0} \frac{f(\mathbf{x} + h\mathbf{e}_i) - f(\mathbf{x})}{h},$$

where $\mathbf{e}_i$ is the i th standard basis vector.

Thus a partial derivative is a directional rate of change in a coordinate direction. The standard basis vector $\mathbf{e}_i$ changes only the i th coordinate, so the limit above asks: what happens to f if we move slightly in the x_i direction and leave all other coordinates fixed?

Example: A Quadratic Function

Let

$$f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T A \mathbf{x}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \quad A = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}.$$

Then

$$f(\mathbf{x}) = \frac{1}{2} \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \frac{1}{2}(x_1^2 + 2x_1x_2 + x_2^2).$$

Therefore

$$\frac{\partial f}{\partial x_1} = x_1 + x_2, \quad \frac{\partial f}{\partial x_2} = x_1 + x_2.$$

This foreshadows a general rule: differentiating a quadratic produces a linear expression.

12.3 Directional Derivatives

Partial derivatives move only in standard coordinate directions. Sometimes we want to move in an arbitrary direction $\mathbf{v} \in \mathbb{R}^n$. The directional derivative of f at $\mathbf{a}$ in the direction $\mathbf{v}$ is

$$D_{\mathbf{v}}f(\mathbf{a}) = \lim_{h \rightarrow 0} \frac{f(\mathbf{a} + h\mathbf{v}) - f(\mathbf{a})}{h},$$

whenever this limit exists.

A directional derivative depends on both the direction and the magnitude of $\mathbf{v}$. If $\mathbf{v} = \alpha\mathbf{u}$, then

$$D_{\mathbf{v}}f(\mathbf{a}) = \alpha D_{\mathbf{u}}f(\mathbf{a}).$$

For this reason, when we want a pure direction, we often use a unit vector $\mathbf{u}$ with $\|\mathbf{u}\| = 1$.

The key connection to differentiability is

$$D_{\mathbf{v}}f(\mathbf{a}) = Df(\mathbf{a})\mathbf{v}.$$

For scalar-valued functions, this becomes

$$D_{\mathbf{v}}f(\mathbf{a}) = (\nabla f(\mathbf{a}))^T \mathbf{v}.$$

So the directional derivative is a dot product between the gradient and the direction vector.

12.4 Differentiability, Derivatives, and the Gradient

In one-variable calculus, the derivative gives the slope of the tangent line. That tangent line is the best local linear approximation to the function. The same idea extends to functions of several variables.

A scalar-valued function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is differentiable at $\mathbf{a}$ if there is a linear function

$$Df(\mathbf{a}) : \mathbb{R}^n \rightarrow \mathbb{R}$$

such that

$$\lim_{\mathbf{h} \rightarrow \mathbf{0}} \frac{f(\mathbf{a} + \mathbf{h}) - f(\mathbf{a}) - Df(\mathbf{a})\mathbf{h}}{\|\mathbf{h}\|} = 0.$$

Here $\mathbf{h}$ is a vector. The term $Df(\mathbf{a})\mathbf{h}$ is the linear approximation to the change in f .

For a scalar-valued differentiable function, the derivative can be represented by a row vector:

$$Df(\mathbf{a}) = \left[\frac{\partial f}{\partial x_1}(\mathbf{a}) \quad \frac{\partial f}{\partial x_2}(\mathbf{a}) \quad \cdots \quad \frac{\partial f}{\partial x_n}(\mathbf{a}) \right].$$

The gradient is the corresponding column vector:

$$\nabla f(\mathbf{a}) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(\mathbf{a}) \\ \frac{\partial f}{\partial x_2}(\mathbf{a}) \\ \vdots \\ \frac{\partial f}{\partial x_n}(\mathbf{a}) \end{bmatrix}.$$

Thus

$$Df(\mathbf{a}) = (\nabla f(\mathbf{a}))^T.$$

The gradient points in the direction of maximum increase. If $\mathbf{u}$ is constrained to be a unit vector, then by Cauchy-Schwarz,

$$D_{\mathbf{u}}f(\mathbf{a}) = (\nabla f(\mathbf{a}))^T \mathbf{u} \leq \|\nabla f(\mathbf{a})\| \|\mathbf{u}\| = \|\nabla f(\mathbf{a})\|.$$

Equality occurs when $\mathbf{u}$ is aligned with $\nabla f(\mathbf{a})$. Therefore $\nabla f(\mathbf{a})$ is the direction of steepest ascent, while $-\nabla f(\mathbf{a})$ is the direction of steepest descent.

12.5 Jacobians for Vector-Valued Functions

For a vector-valued function

$$f : \mathbb{R}^n \rightarrow \mathbb{R}^m,$$

partial derivatives are vectors in $\mathbb{R}^m$. The derivative is the matrix obtained by stacking these partial derivative vectors as columns:

$$Df(\mathbf{a}) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(\mathbf{a}) & \frac{\partial f}{\partial x_2}(\mathbf{a}) & \cdots & \frac{\partial f}{\partial x_n}(\mathbf{a}) \end{bmatrix} \in \mathbb{R}^{m \times n}.$$

This matrix is called the *Jacobian matrix*.

For example, let

$$f(\mathbf{x}) = \begin{bmatrix} x_1^2 + x_2^2 \\ x_1 + x_2 + x_1x_2 \end{bmatrix}.$$

Then

$$\frac{\partial f}{\partial x_1} = \begin{bmatrix} 2x_1 \\ 1 + x_2 \end{bmatrix}, \quad \frac{\partial f}{\partial x_2} = \begin{bmatrix} 2x_2 \\ 1 + x_1 \end{bmatrix},$$

so

$$Df(\mathbf{x}) = \begin{bmatrix} 2x_1 & 2x_2 \\ 1 + x_2 & 1 + x_1 \end{bmatrix}.$$

At $\mathbf{x} = (1, 1)^T$, the directional derivative in the direction $\mathbf{v} = (1, 1)^T$ is

$$D_{\mathbf{v}}f(\mathbf{x}) = Df(\mathbf{x})\mathbf{v} = \begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 4 \end{bmatrix}.$$

The output is a vector because the function itself has vector output.

12.6 Derivative Rules

The source notes emphasize a few derivative rules that are especially useful for linear algebra and optimization.

Linear functions

If

$$f(\mathbf{x}) = A\mathbf{x}, \quad A \in \mathbb{R}^{m \times n},$$

then

$$Df(\mathbf{x}) = A.$$

The derivative of a linear transformation is the matrix itself.

Quadratic functions

If

$$f(\mathbf{x}) = \mathbf{x}^T P \mathbf{x},$$

where $P \in \mathbb{R}^{n \times n}$, then

$$Df(\mathbf{x}) = \mathbf{x}^T (P + P^T),$$

and

$$\nabla f(\mathbf{x}) = (P + P^T)\mathbf{x}.$$

If P is symmetric, this simplifies to

$$\nabla f(\mathbf{x}) = 2P\mathbf{x}.$$

Inner products

If $\mathbf{v} \in \mathbb{R}^n$ is constant and

$$f(\mathbf{x}) = \mathbf{x}^T \mathbf{v} = \mathbf{v}^T \mathbf{x},$$

then

$$Df(\mathbf{x}) = \mathbf{v}^T, \quad \nabla f(\mathbf{x}) = \mathbf{v}.$$

Product rule

If $f, g : \mathbb{R}^n \rightarrow \mathbb{R}$, then

$$D(fg)(\mathbf{a})\mathbf{v} = Df(\mathbf{a})\mathbf{v}g(\mathbf{a}) + f(\mathbf{a})Dg(\mathbf{a})\mathbf{v}.$$

This is the multivariable version of the one-variable rule $(fg)' = f'g + fg'$.

12.7 The Chain Rule

The chain rule is the most important rule for differentiating composed functions. Suppose

$$g : \mathbb{R}^n \rightarrow \mathbb{R}^m, \quad f : \mathbb{R}^m \rightarrow \mathbb{R}^k.$$

Then the composition $f \circ g : \mathbb{R}^n \rightarrow \mathbb{R}^k$ is defined by

$$(f \circ g)(\mathbf{x}) = f(g(\mathbf{x})).$$

If both functions are differentiable, then

$$D(f \circ g)(\mathbf{a}) = Df(g(\mathbf{a}))Dg(\mathbf{a}).$$

The dimensions line up as

$$Df(g(\mathbf{a})) \in \mathbb{R}^{k \times m}, \quad Dg(\mathbf{a}) \in \mathbb{R}^{m \times n},$$

so

$$D(f \circ g)(\mathbf{a}) \in \mathbb{R}^{k \times n}.$$

Example: Distance from a Radar Tower

Suppose an airplane is near a radar tower at the origin. Let

$$\mathbf{g}(t) = \begin{bmatrix} g_1(t) \\ g_2(t) \\ g_3(t) \end{bmatrix}$$

be the aircraft's position at time t , where g_1 measures miles east of the tower, g_2 measures miles north of the tower, and g_3 measures height in miles. The distance from the tower is

$$f(\mathbf{g}(t)), \quad f(\mathbf{x}) = \sqrt{x_1^2 + x_2^2 + x_3^2}.$$

By the chain rule,

$$D(f \circ \mathbf{g})(t) = Df(\mathbf{g}(t))D\mathbf{g}(t).$$

For $f(\mathbf{x}) = \|\mathbf{x}\|$,

$$Df(\mathbf{x}) = \left[\frac{x_1}{\sqrt{x_1^2 + x_2^2 + x_3^2}} \quad \frac{x_2}{\sqrt{x_1^2 + x_2^2 + x_3^2}} \quad \frac{x_3}{\sqrt{x_1^2 + x_2^2 + x_3^2}} \right].$$

At the instant in the notes, the aircraft is 3 miles west of the tower and 4 miles high, so

$$\mathbf{g}(t) = \begin{bmatrix} -3 \\ 0 \\ 4 \end{bmatrix}.$$

It is flying due east at 450 miles per hour and climbing at 5 miles per hour, so

$$D\mathbf{g}(t) = \begin{bmatrix} 450 \\ 0 \\ 5 \end{bmatrix}.$$

Thus

$$D(f \circ \mathbf{g})(t) = \frac{1}{5} \begin{bmatrix} -3 & 0 & 4 \end{bmatrix} \begin{bmatrix} 450 \\ 0 \\ 5 \end{bmatrix} = \frac{-1350 + 20}{5} = -266.$$

The distance is changing at -266 miles per hour. The negative sign means the plane is getting closer to the tower at that instant.

12.8 Stationary Points

For a scalar-valued function

$$f : \mathbb{R}^n \rightarrow \mathbb{R},$$

a point $\mathbf{x} \in \mathbb{R}^n$ is called a *stationary point* if

$$\nabla f(\mathbf{x}) = \mathbf{0}.$$

This is the multivariable analogue of solving $f'(x) = 0$ in one-variable calculus.

Local minimizers and local maximizers, when they occur in the interior of the domain and the function is differentiable, must be stationary points. However, not every stationary point is a minimum or maximum. A stationary point may also be a saddle point. Chapter 13 develops the Hessian and second-order tests that help distinguish these cases. In this chapter, we use the stationary-point condition to derive the least-squares normal equations.

12.9 The Least-Squares Problem

Suppose

$$A \in \mathbb{R}^{m \times n}, \quad \mathbf{b} \in \mathbb{R}^m,$$

and suppose A is tall, with linearly independent columns. The system

$$A\mathbf{x} = \mathbf{b}$$

may have no exact solution because $\mathbf{b}$ may not lie in $\text{Col}(A)$. Least squares changes the question from exact solution to best approximation:

$$\min_{\mathbf{x} \in \mathbb{R}^n} \|A\mathbf{x} - \mathbf{b}\|^2.$$

The residual vector for a candidate $\mathbf{x}$ is

$$\mathbf{r}(\mathbf{x}) = A\mathbf{x} - \mathbf{b}.$$

The least-squares objective is the squared residual length:

$$\|A\mathbf{x} - \mathbf{b}\|^2.$$

Since a norm is always nonnegative, the best possible case is $\|A\mathbf{x} - \mathbf{b}\|^2 = 0$, which means the original system has an exact solution. When no exact solution exists, least squares chooses the $\mathbf{x}$ that makes the residual as small as possible.

12.10 Expanding the Least-Squares Objective

Expand the objective:

$$\begin{aligned} \|A\mathbf{x} - \mathbf{b}\|^2 &= (A\mathbf{x} - \mathbf{b})^T (A\mathbf{x} - \mathbf{b}) \\ &= (\mathbf{x}^T A^T - \mathbf{b}^T)(A\mathbf{x} - \mathbf{b}) \\ &= \mathbf{x}^T A^T A\mathbf{x} - \mathbf{x}^T A^T \mathbf{b} - \mathbf{b}^T A\mathbf{x} + \mathbf{b}^T \mathbf{b}. \end{aligned}$$

The middle two terms are scalars, and

$$\mathbf{x}^T A^T \mathbf{b} = \mathbf{b}^T A\mathbf{x}.$$

Therefore

$$\|A\mathbf{x} - \mathbf{b}\|^2 = \mathbf{x}^T A^T A\mathbf{x} - 2\mathbf{b}^T A\mathbf{x} + \mathbf{b}^T \mathbf{b}.$$

This is a quadratic function of $\mathbf{x}$. The last term $\mathbf{b}^T \mathbf{b}$ does not depend on $\mathbf{x}$, so it does not affect which $\mathbf{x}$ minimizes the objective.

12.11 Gradient and the Normal Equations

Using the derivative rules above, the gradient of the quadratic term is

$$\nabla_{\mathbf{x}}(\mathbf{x}^T A^T A\mathbf{x}) = 2A^T A\mathbf{x},$$

because $A^T A$ is symmetric. The gradient of the linear term is

$$\nabla_{\mathbf{x}}(-2\mathbf{b}^T A\mathbf{x}) = -2A^T \mathbf{b}.$$

Thus

$$\nabla_{\mathbf{x}}\|A\mathbf{x} - \mathbf{b}\|^2 = 2A^T A\mathbf{x} - 2A^T \mathbf{b}.$$

At a stationary point, this gradient is zero:

$$2A^T A\mathbf{x} - 2A^T \mathbf{b} = \mathbf{0}.$$

Dividing by 2 gives

$$A^T A\mathbf{x} = A^T \mathbf{b}.$$

These are the *normal equations*.

If the columns of A are linearly independent, then $A^T A$ is invertible. Therefore the least-squares solution is

$$\hat{\mathbf{x}} = (A^T A)^{-1} A^T \mathbf{b}.$$

Using the full-column-rank left-pseudoinverse notation from Chapter 10,

$$A_L^+ = (A^T A)^{-1} A^T,$$

so

$$\hat{\mathbf{x}} = A_L^+ \mathbf{b}.$$

Chapter 13 explains the second-order viewpoint behind why this stationary point is a minimum. For now, the key point is that full column rank gives a unique least-squares solution.

12.12 Residual Orthogonality and Projection

The normal equations can be rewritten as

$$A^T (A\hat{\mathbf{x}} - \mathbf{b}) = \mathbf{0}.$$

Using $\mathbf{r} = A\hat{\mathbf{x}} - \mathbf{b}$, this says

$$A^T \mathbf{r} = \mathbf{0}.$$

So the least-squares residual is orthogonal to every column of A .

In many applications, including regression, it is more natural to define the error vector with the opposite sign:

$$\mathbf{e} = \mathbf{b} - A\hat{\mathbf{x}}.$$

Then

$$A^T \mathbf{e} = \mathbf{0}$$

as well. Changing the sign of the residual does not change orthogonality.

This gives the geometric interpretation of least squares. The vector $A\hat{\mathbf{x}}$ lies in the column space of A . Least squares chooses the vector in $\text{Col}(A)$ that is closest to $\mathbf{b}$. The difference between $\mathbf{b}$ and that closest vector is orthogonal to the column space.

In words:

Least squares projects $\mathbf{b}$ onto the column space of A .

This is the mathematical foundation for the regression statement that fitted values live in the column space of the design matrix and residuals are orthogonal to the design columns.

12.13 A Chain-Rule Application: Neural-Network Loss

The source notes use a neural-network example to show why the multivariable chain rule matters computationally. Let

$$f: \mathbb{R}^n \rightarrow \mathbb{R}^t$$

be a model output, and let $\mathbf{c} \in \mathbb{R}^t$ be a target vector. Define the squared-error loss

$$g(\mathbf{x}) = \|f(\mathbf{x}) - \mathbf{c}\|^2.$$

Let

$$u(\mathbf{z}) = \|\mathbf{z} - \mathbf{c}\|^2.$$

Then

$$g = u \circ f.$$

The derivative of u is

$$Du(\mathbf{z}) = 2(\mathbf{z} - \mathbf{c})^T.$$

By the chain rule,

$$Dg(\mathbf{x}) = Du(f(\mathbf{x}))Df(\mathbf{x}) = 2(f(\mathbf{x}) - \mathbf{c})^T Df(\mathbf{x}).$$

For a network made of alternating linear transformations and elementwise nonlinearities, such as

$$f(\mathbf{x}) = W_4\sigma(W_3\sigma(W_2\sigma(W_1\mathbf{x}))),$$

the derivative $Df(\mathbf{x})$ is a product of Jacobians. The derivative of a linear layer is its weight matrix. The derivative of an elementwise activation σ is a diagonal matrix whose entries are the scalar derivatives $\sigma'(z_i)$:

$$D\sigma(\mathbf{z}) = \begin{bmatrix} \sigma'(z_1) & 0 & \cdots & 0 \\ 0 & \sigma'(z_2) & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma'(z_n) \end{bmatrix}.$$

For the ReLU activation,

$$\sigma(z) = \max(0, z),$$

the derivative is 1 when the unit is active and 0 when it is inactive, ignoring the nondifferentiable point at zero for this introductory discussion.

The important point is not the neural-network architecture itself. The important point is that the same derivative bookkeeping used in least squares also underlies modern gradient-based fitting: define a scalar loss, compute its gradient, and use that gradient to update the inputs or parameters.

12.14 Data Analysis Bridge

In the Data Analysis textbook, least squares regression is written with a design matrix X , response vector $\mathbf{y}$, coefficient vector $\boldsymbol{\beta}$, fitted vector $X\hat{\boldsymbol{\beta}}$, and residual vector

$$\mathbf{e} = \mathbf{y} - X\hat{\boldsymbol{\beta}}.$$

This chapter explains why the coefficient estimate satisfies

$$X^T X \hat{\boldsymbol{\beta}} = X^T \mathbf{y},$$

and why

$$X^T \mathbf{e} = \mathbf{0}.$$

The first equation is the normal equation. The second equation is residual orthogonality. Together they say that least squares chooses the fitted vector in $\text{Col}(X)$ closest to the observed response vector.

This is the same projection idea that appears in regression geometry. The only difference is notation: in this chapter the generic matrix is A and the generic right-hand side is $\mathbf{b}$; in regression, A becomes X and $\mathbf{b}$ becomes $\mathbf{y}$.

12.15 Chapter Summary

The main ideas of this chapter are:

- A partial derivative changes one coordinate while holding the others fixed.
- A directional derivative changes the input in an arbitrary vector direction.
- For scalar-valued functions, the gradient is the column vector of partial derivatives and points in the direction of steepest ascent.
- For vector-valued functions, the derivative is the Jacobian matrix whose columns are partial derivative vectors.
- The chain rule multiplies Jacobians in the order required by the composition.
- A stationary point of a differentiable scalar-valued function satisfies $\nabla f(\mathbf{x}) = \mathbf{0}$.
- Least squares minimizes $\|A\mathbf{x} - \mathbf{b}\|^2$.
- Expanding the least-squares objective and setting its gradient to zero gives the normal equations $A^T A \hat{\mathbf{x}} = A^T \mathbf{b}$.
- When A has linearly independent columns, the least-squares solution is $\hat{\mathbf{x}} = (A^T A)^{-1} A^T \mathbf{b}$.
- The least-squares residual is orthogonal to the columns of A , so least squares is projection onto the column space of A .